Final Projects and LLMs

Hrachya Asatryan

Final Projects

- **Target Code Freeze:** Wednesday, August 26th
- **Final Repo Lock:** Sunday, August 30th @ 11:59 PM (GitHub Link submitted to me)
- **Presentation Day:** Wednesday, September 2nd

Note on pacing: I strongly advise you to have your model fully trained and your code "frozen" by August 26th. Building a good Deep Learning model is only half the battle; analyzing where it failed and building a compelling 10-minute presentation takes time. The final GitHub link is not due until Sunday night to give you a weekend buffer for bug fixes. Do not leave training until the last minute.

Final Projects

- **This is not a homework assignment; this is a portfolio piece.**
- You are to take your assigned dataset, build a deep learning pipeline, and present your findings like a professional Data Science team reporting to stakeholders.
- You will be graded on two main deliverables:
  1. The GitHub Repository (Your Code)
  2. The Presentation (Your Communication)

Final Projects

Deliverable 1: The GitHub Repository (Due Aug 30)

I do not want a zip file of a messy Jupyter notebook. You must submit a link to a public, professional GitHub repository.

- **A Professional README.md:** This is the front page. It must include the Project Overview, Architecture used, Results (with 1-2 visual graphs), and instructions on how to run your code.
- **Clean Code Structure:**
  - Do NOT upload the dataset to GitHub (use .gitignore).
  - Notebooks must be clean, heavily commented, and run top-to-bottom without errors.
  - *(Bonus points for modularizing code into dataset.py, train.py, model.py)*
  - Include a requirements.txt.

Final Projects

Deliverable 1: The Inference Script (predict.py)

Training a model is only half the job. You must include a standalone script (or a clearly marked section in your notebook) dedicated entirely to inference on unseen data.

- **Load the Saved Model:** Load your best saved weights without retraining.
- **Handle Raw Input:** Accept a single raw file path (image) or raw string (text).
- **Apply Exact Preprocessing:** Apply the same transforms/tokenization used during training.
- **Output Human-Readable Results:** Decode the output! No raw tensor logits `tensor([[0.12, 0.88]])`.
  - *Example:* "Prediction: Malignant Melanoma | Confidence: 94.2%"
- **The "Real-World" Test:** I should be able to point your script to a random leaf picture or random tweet and get a prediction in seconds.

Final Projects

Deliverable 2: The Presentation (Sept 2)

Each group gets exactly 10 to present, plus 2 minutes of Q&A. **Every team member must speak.** Your deck must cover 5 pillars:

1. **The Problem & The Data (2 mins):** What is the task? Show examples. Were there class imbalances or outliers?
2. **Data Preprocessing (2 mins):** How did you prepare the data? (e.g., resizing, augmentations, tokenization).
3. **The Baseline Model (2 mins):** What happened when you ran a simple CNN or a basic RNN/Linear layer before building your massive model?
4. **The Final Architecture (2 mins):** What is your final model? Did you use Transfer Learning? Why did you choose it?
5. **Results & Error Analysis (2 mins):** *The most important part.* Show your metrics, but more importantly, show us where the model **FAILED**. Show 2-3 incorrect predictions and explain *why* the network got confused. Prove you understand the data!

Final Projects

Group 1: NLP - Student Writing Evaluation

- **Members:** Onik Mantashyan, Artyom Ghulyan, Veronica Terteryan, Alen Gasparyan
- **Dataset:** Feedback Prize - Evaluating Student Writing
- **Link:** https://www.kaggle.com/competitions/feedback-prize-2021
- **The Setup:** Standard classification is too easy. Your task is **Token Classification / Named Entity Recognition**. You are given full argumentative essays written by students. You must build an NLP pipeline that segments the text and classifies each section into discourse elements (e.g., Lead, Position, Claim, Counterclaim, Rebuttal).

Final Projects

Group 2: Multimodal - Product Matching

- **Members:** Davit Bichaxchyan, Hovhannes Apoyan, Ara Davtyan
- **Dataset:** Shopee - Price Match Guarantee
- **Link:** https://www.kaggle.com/c/shopee-product-matching
- **The Setup:** E-commerce platforms struggle with sellers posting the exact same item under different names and photos. You must build a **Multimodal** model that takes in both the product image (CV) and the text title/description (NLP) to predict if two listings are actually the exact same product.

Final Projects

Group 3: CV Challenge - Steel Defect Detection

- **Members:** Narek Gabrielyan, Gor Tonoyan
- **Dataset:** Severstal: Steel Defect Detection
- **Link:** https://www.kaggle.com/c/severstal-steel-defect-detection
- **The Setup:** Simple image classification isn't enough here. You are tasked with **Semantic Segmentation**. You must build a model that looks at images of sheet steel, classifies the type of defect present, and generates a pixel-perfect mask locating exactly where the defect is on the metal.

Final Projects

Group 4: CV - Cassava Leaf Disease

- **Members:** Syuzanna Makaryan, Viktorya Margaryan, Ruzanna Torosyan, Christine Mkhitaryan
- **Dataset:** Cassava Leaf Disease Classification
- **Link:** https://www.kaggle.com/competitions/cassava-leaf-disease-classification
- **The Setup:** Cassava is a staple food for over half a billion people in Africa, but viral diseases can devastate crops. Using a dataset of over 21,000 images taken by farmers, you must build a robust Computer Vision pipeline to classify which of 4 diseases (or healthy) is present on the leaf.

Final Projects

Group 5: CV - Global Wheat Detection

- **Members:** Ruzanna Mkhitaryan, Anahit Mkrtumyan, Eva Araratyan, Gor Papoyan
- **Dataset:** Global Wheat Detection
- **Link:** https://www.kaggle.com/competitions/global-wheat-detection
- **The Setup:** To estimate crop yields, agriculturists need to know exactly how many wheat heads are in a field. You must build an **Object Detection** model. Rather than just classifying an image, your model must draw accurate bounding boxes around every single wheat head visible in an image of a dense crop field.

Final Projects

Group 6: CV - Histopathologic Cancer Detection

- **Members:** Garik Ghazaryan, Hayk Grigoryan, Alexander Hajoyan, Gohar Yepremyan
- **Dataset:** Histopathologic Cancer Detection
- **Link:** https://www.kaggle.com/competitions/histopathologic-cancer-detection
- **The Setup:** You are tasked with analyzing microscopic tissue scans to aid pathologists. You must build a binary classification model that looks at small, highly detailed image patches extracted from larger digital pathology scans and predicts whether or not metastatic cancer tissue is present in the center of that patch.

Final Projects

Group 7: CV - Melanoma Classification

- **Members:** Ruzanna Barseghyan, Anahit Tumasyan, Mariam Petrosyan, Harutyun Qesablyan
- **Dataset:** SIIM-ISIC Melanoma Classification
- **Link:** https://www.kaggle.com/competitions/siim-isic-melanoma-classification
- **The Setup:** Melanoma is a deadly but highly curable form of skin cancer if caught early. You are provided with images of skin lesions alongside patient-level metadata. Your goal is to build an image classification pipeline to identify which skin lesions are malignant melanomas and which are benign.

Final Projects

Group 8: CV - Diabetic Retinopathy Blindness Detection

- **Members:** Arman Kareyan, Aramayis Mesropyan, Vahan Sargsyan, Elen Hakobyan
- **Dataset:** APTOS 2019 Blindness Detection
- **Link:** https://www.kaggle.com/c/aptos2019-blindness-detection
- **The Setup:** Diabetic retinopathy is a leading cause of blindness, but can be prevented with early screening. You will use a dataset of high-resolution retina images taken under various imaging conditions to build a model that classifies the severity of diabetic retinopathy on a scale of 0 (No DR) to 4 (Proliferative DR).

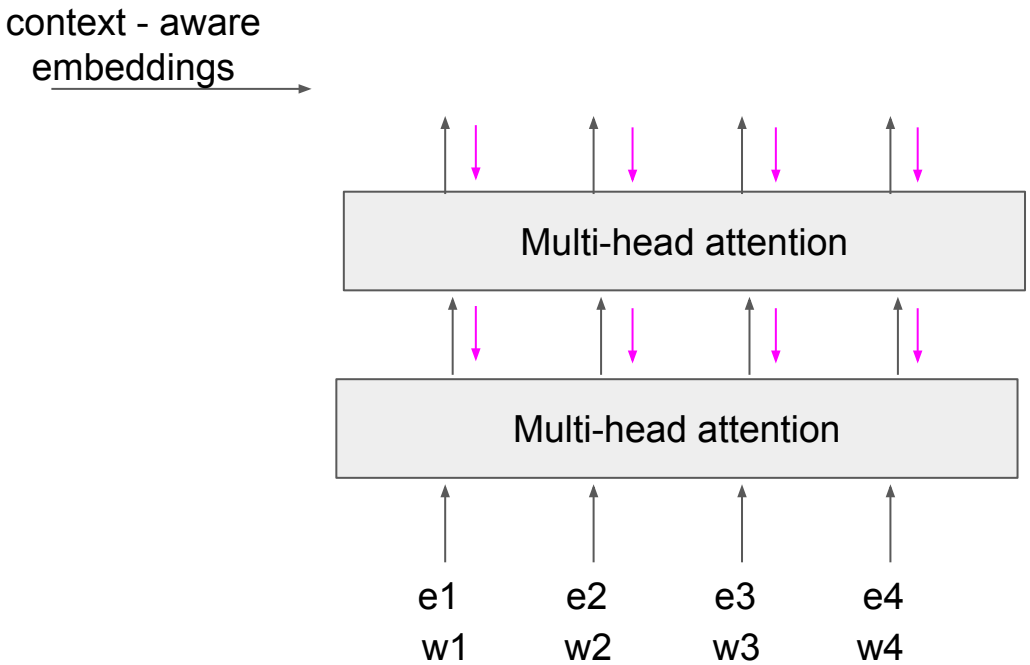
Final Projects

Group 9: CV - Plant Pathology 2021

- **Members:** Erik Bagdasaryan, Samvel Fahradyan, Anastasiya Tikunova, Aram Khachatryan
- **Dataset:** Plant Pathology 2021 - FGVC8
- **Link:** https://www.kaggle.com/c/plant-pathology-2021-fgvc8
- **The Setup:** Apples are one of the most important temperate fruit crops globally, but their yield is constantly threatened by foliar diseases. You must build a robust image classification model to accurately identify different diseases in apple leaves, handling complex scenarios like leaves with multiple diseases at once or varying background lighting.

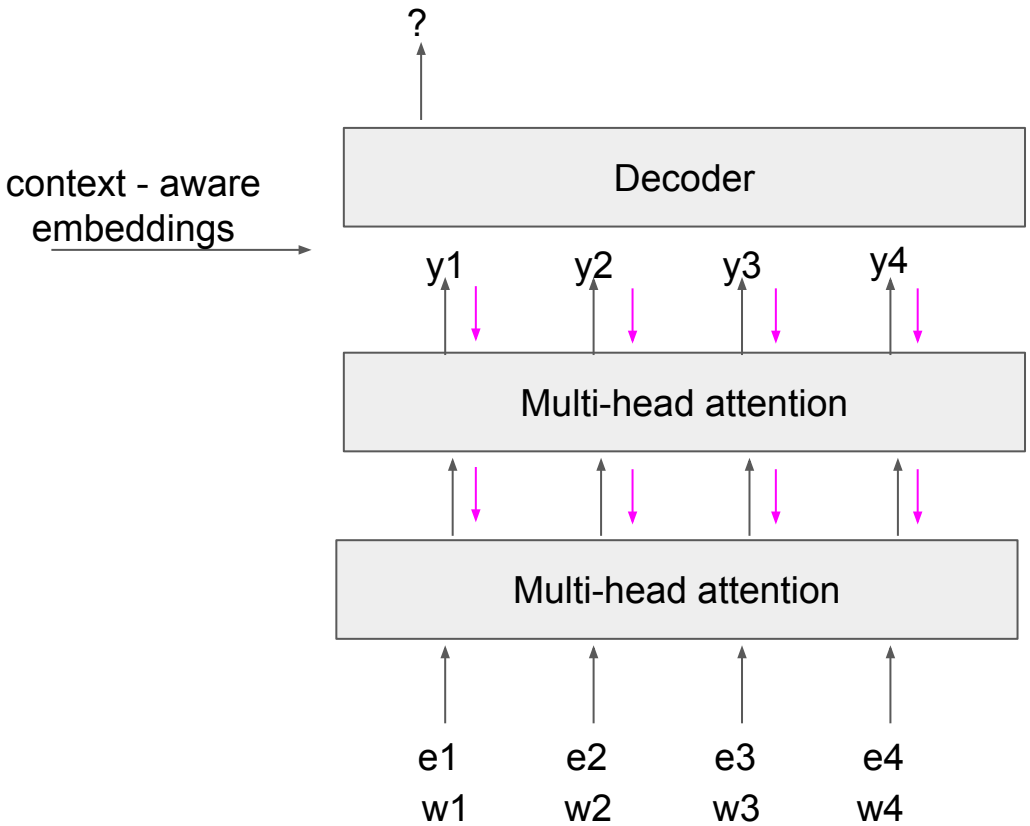
What tasks can we use transformers for?

Task?



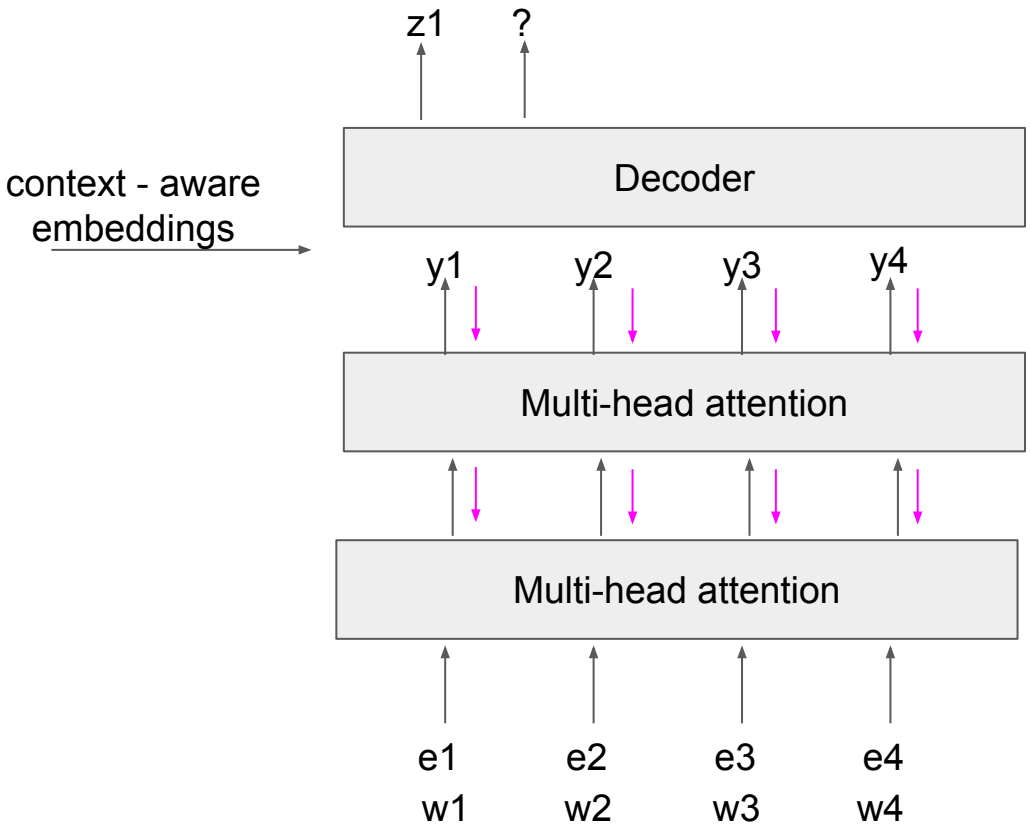
What tasks can we use transformers for?

Seq2seq (e.g. translation, question answering, summarization, ...)



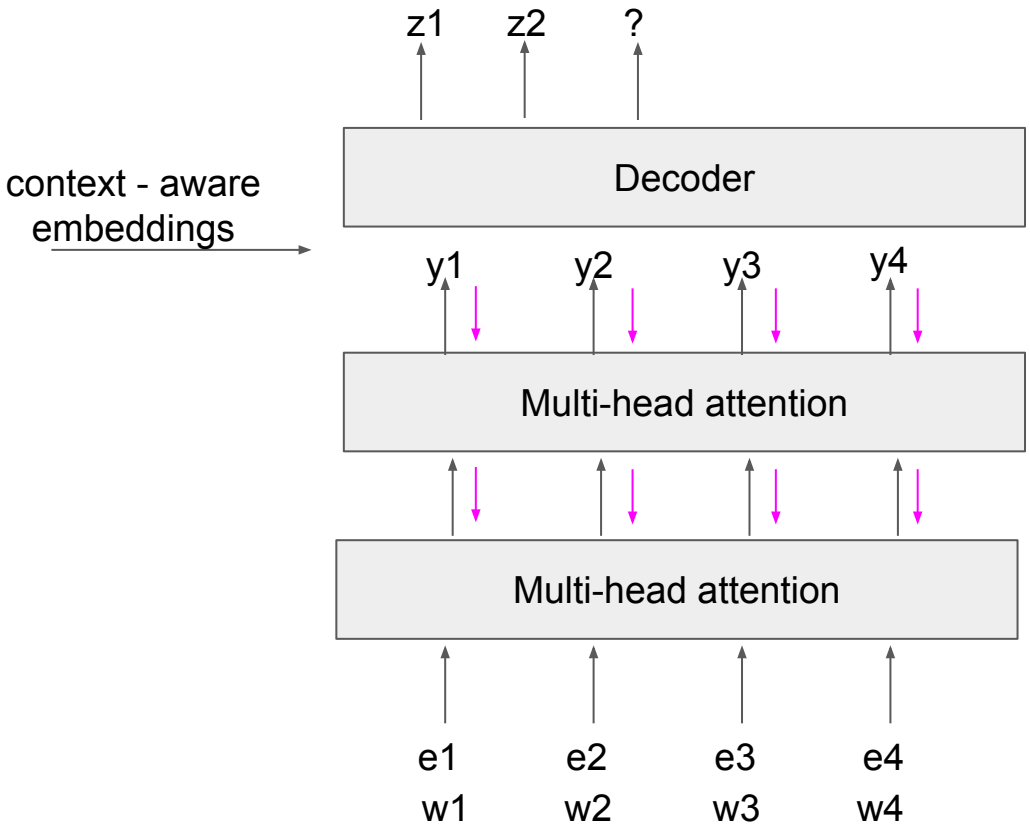
What tasks can we use transformers for?

Seq2seq (e.g. translation, question answering, summarization, ...)



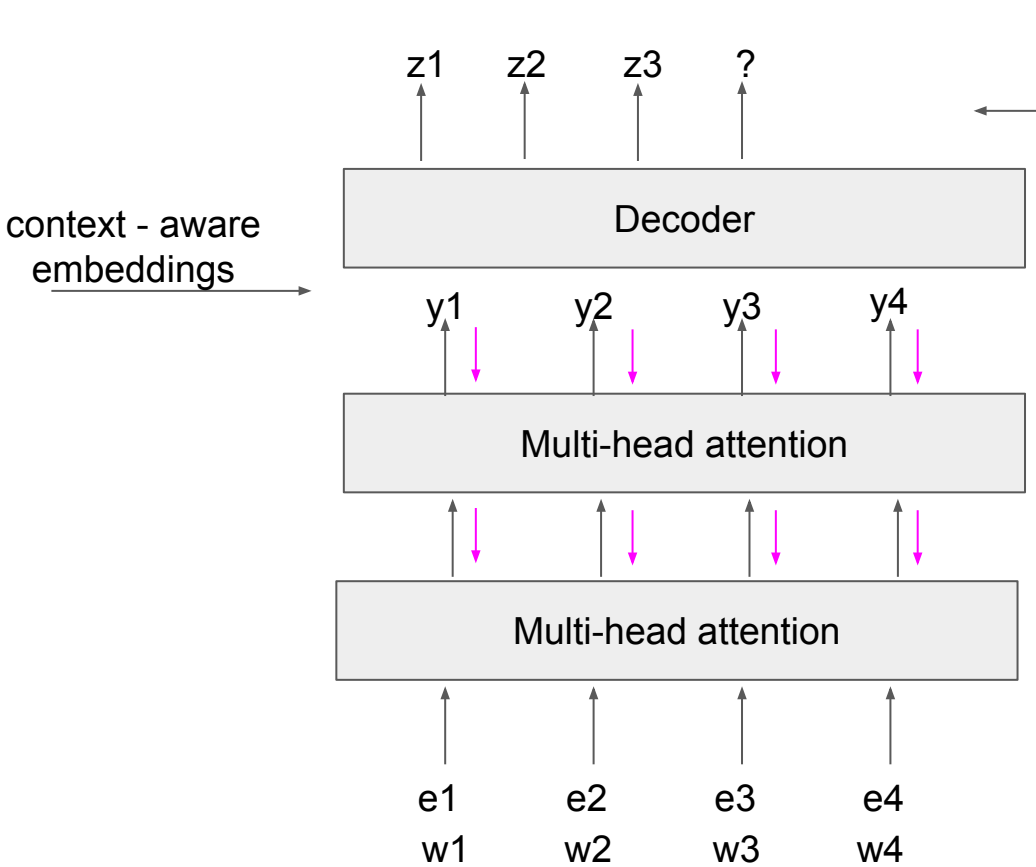
What tasks can we use transformers for?

Seq2seq (e.g. translation, question answering, summarization, ...)



What tasks can we use transformers for?

Seq2seq (e.g. translation, question answering, summarization,...)



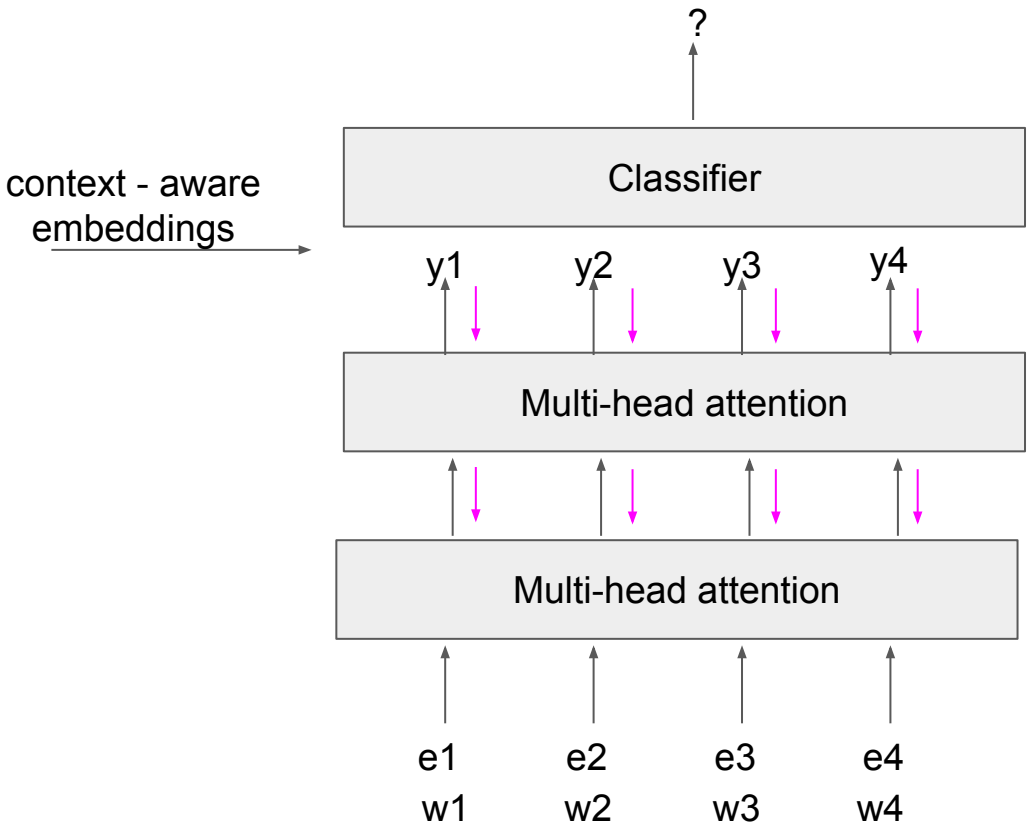
Supervised!
Need a lot of **labeled** data!

(embeddings would be very task-dependent)

What tasks can we use transformers for?

Classifier (e.g. sentiment analysis, document classification)

single output

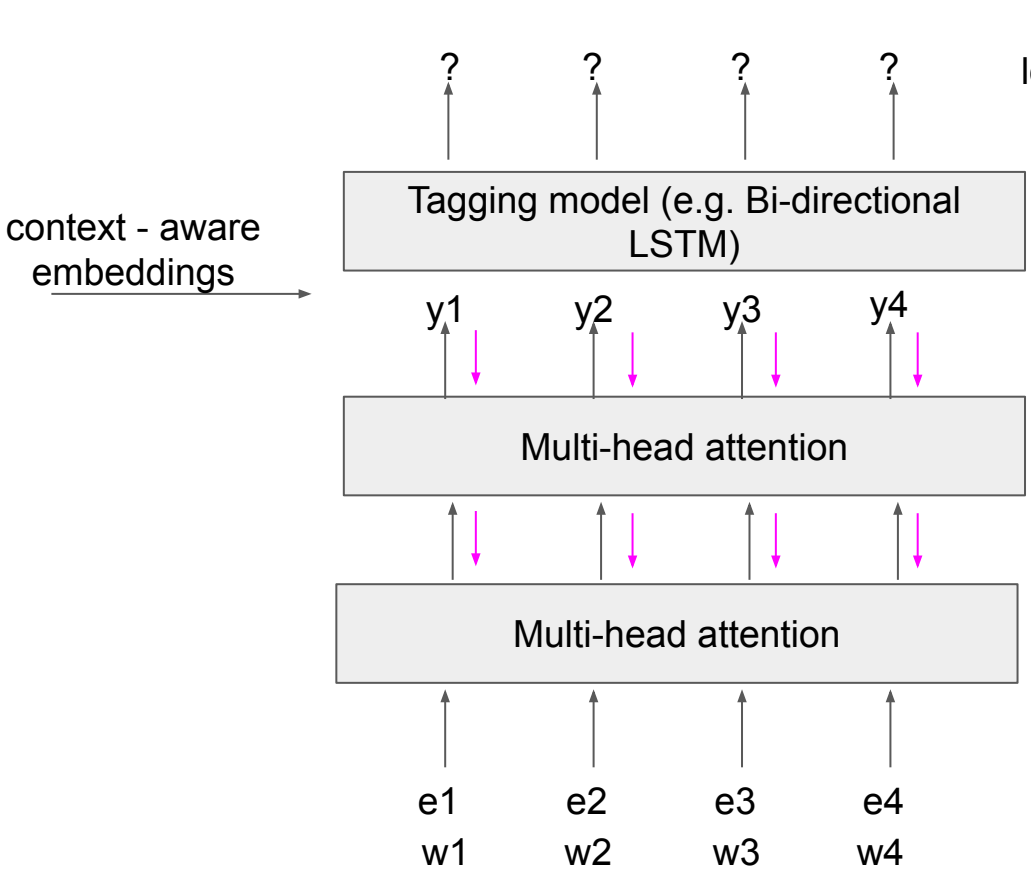


Supervised!
Need a lot of **labeled** data!

(embeddings would be very task-dependent)

What tasks can we use transformers for?

Sequence tagging (e.g. part of speech, named entity recognition, …)

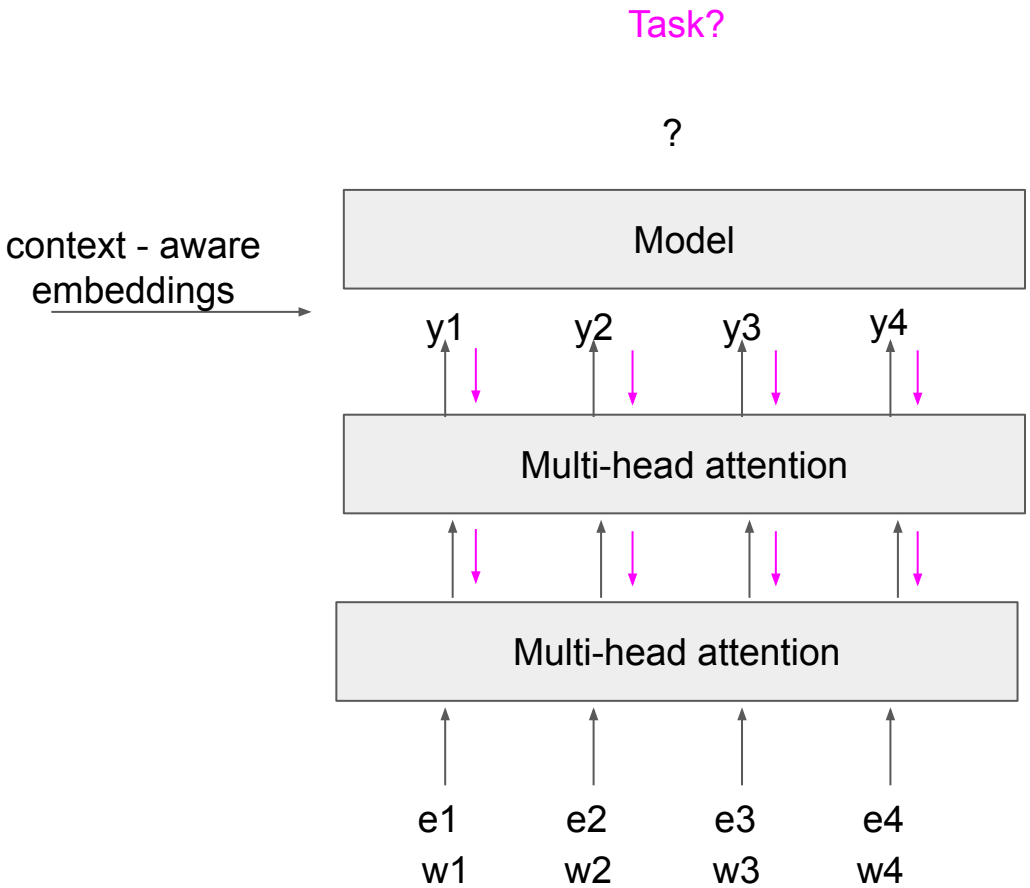


generate all at once, the length of the output is the same as input

Supervised!
Need a lot of **labeled** data!

(embeddings would be very task-dependent)

What tasks can we use transformers for?



Can we find a self-supervised task to pre-train 'universal' embeddings that we can use for anything later on?

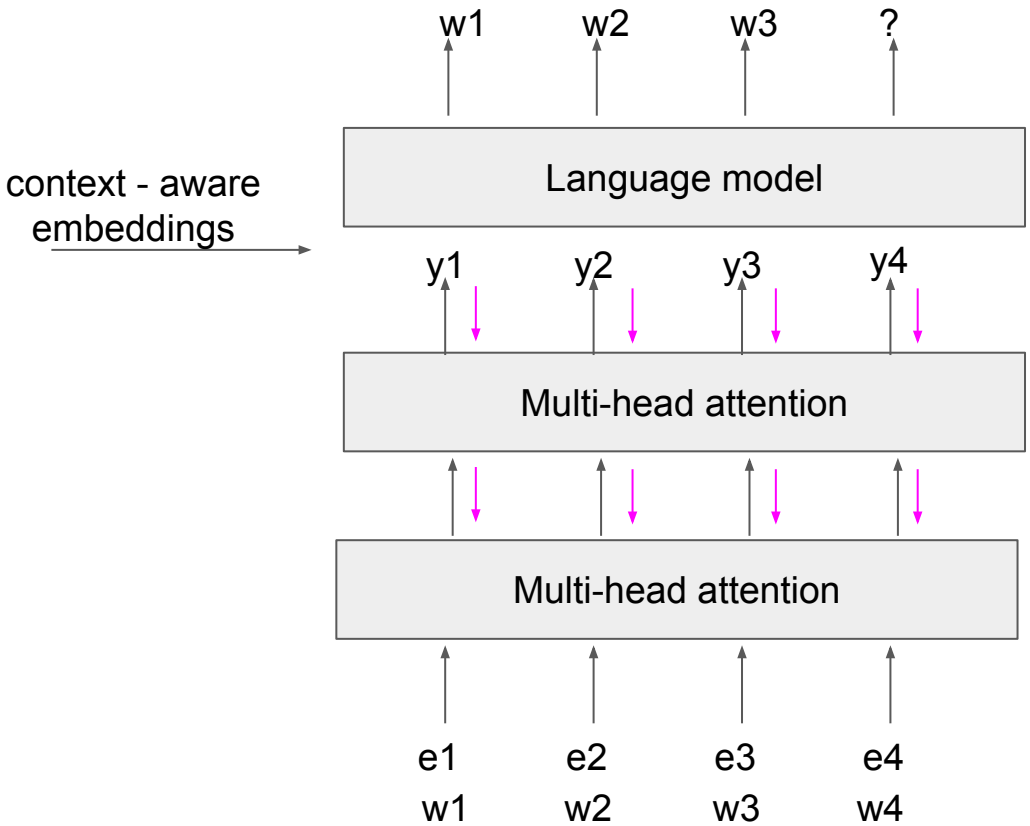
The Transfer Learning Revolution

- **The NLP Initialization Problem:** Before 2018, every time we built a Sentiment Classifier or Translator, we initialized the network with random weights and trained it from scratch on a few thousand labeled examples.
- **The Bottleneck:** A few thousand examples are not enough to teach a neural network the entirety of English grammar, logic, and reasoning.
- **The Solution (Self-Supervised Learning):** The internet is the dataset. We do not need human labelers. We can take terabytes of raw text (Wikipedia, Reddit), mathematically hide parts of it, and force a massive Transformer to guess the missing pieces.
- By doing this billions of times, the network learns world knowledge natively. Once pre-trained, we freeze the core network and only train a tiny final layer for our specific task (Fine-Tuning).

What tasks can we use transformers for?

Language model

generate one at a time

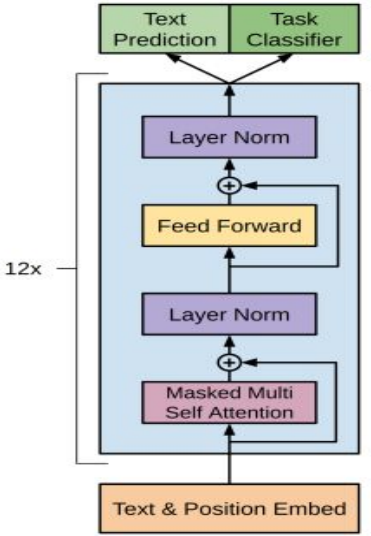


Self-Supervised!
Do not need labelled data!

(embeddings could be
universal)

GPT-1

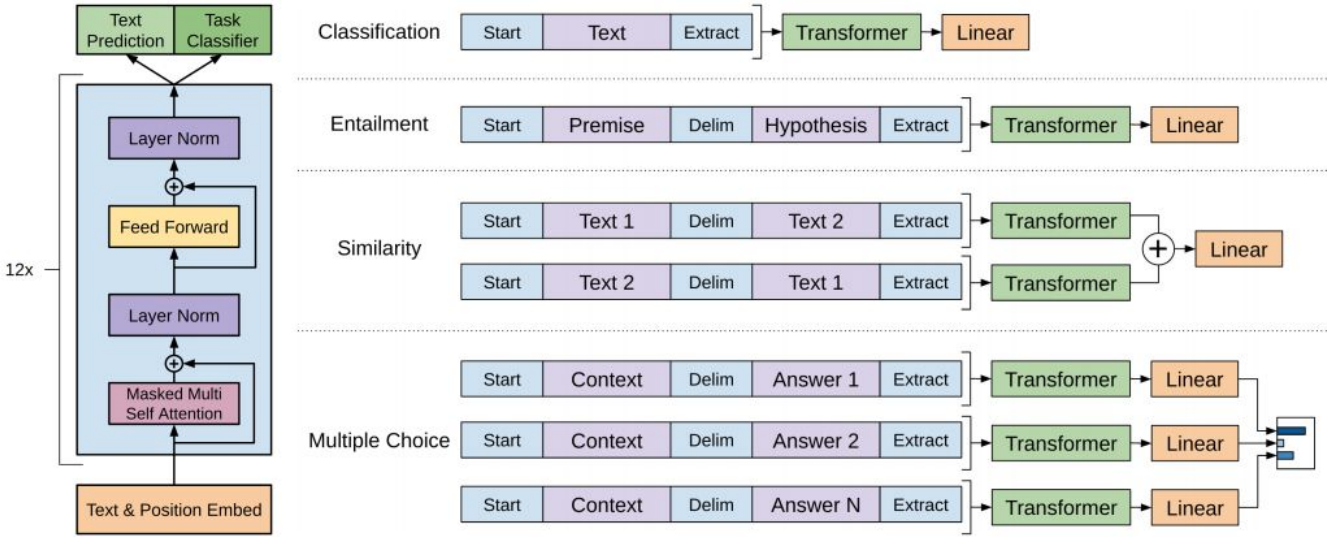
- **The Architecture:** In 2018, OpenAI took the original Transformer and removed the Encoder entirely. GPT is a **Decoder-Only** network.
- **Masked Self-Attention:** Unlike bidirectional models, GPT enforces a strict left-to-right reading order. When processing token X, the attention matrix is "masked" (future tokens are set to -∞), making it mathematically impossible for the network to look ahead.
- **The Objective:** Because it cannot see the future, it is trained on pure **Next-Token Prediction**.
- *Example:* "The first president of the United States was ___". To minimize its loss function and correctly predict "George", the network is forced to natively learn history, grammar, and facts hidden within the training data.



GPT-1

Improving Language Understanding by Generative Pre-Training
Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever
https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

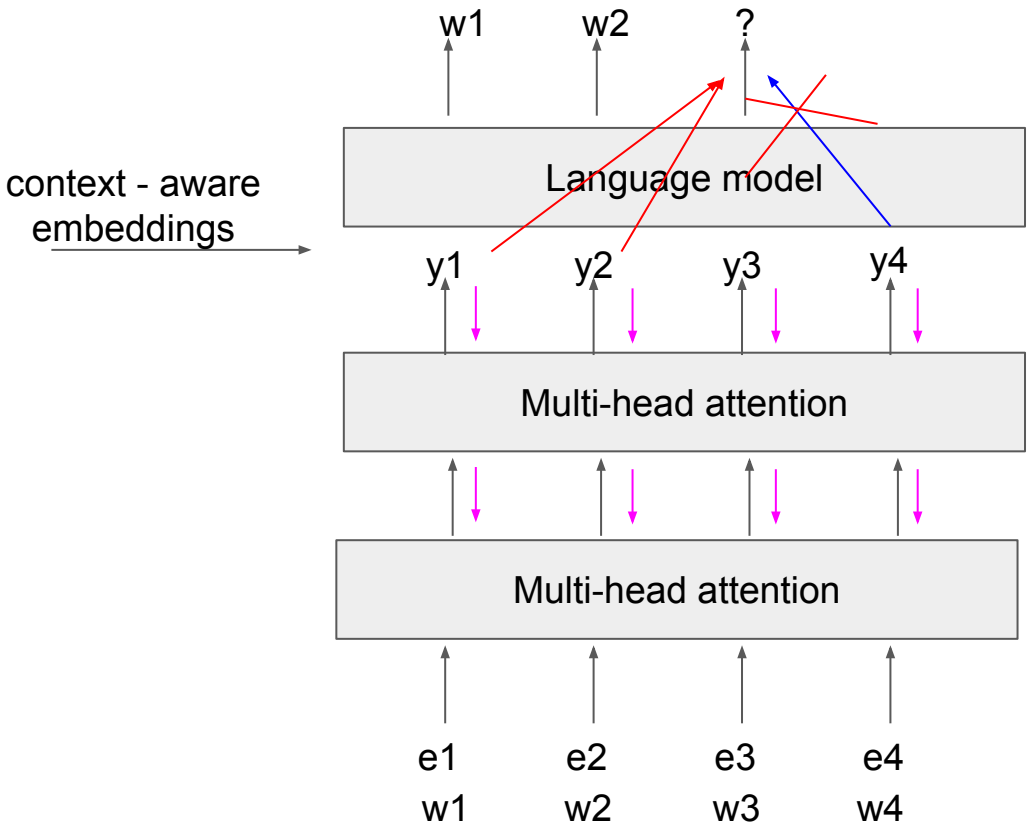
Fine-tuning the pre-trained model on more meaningful tasks with labelled data



Drawbacks

Language model

generate one at a time



Self-Supervised!
Do not need labelled data!

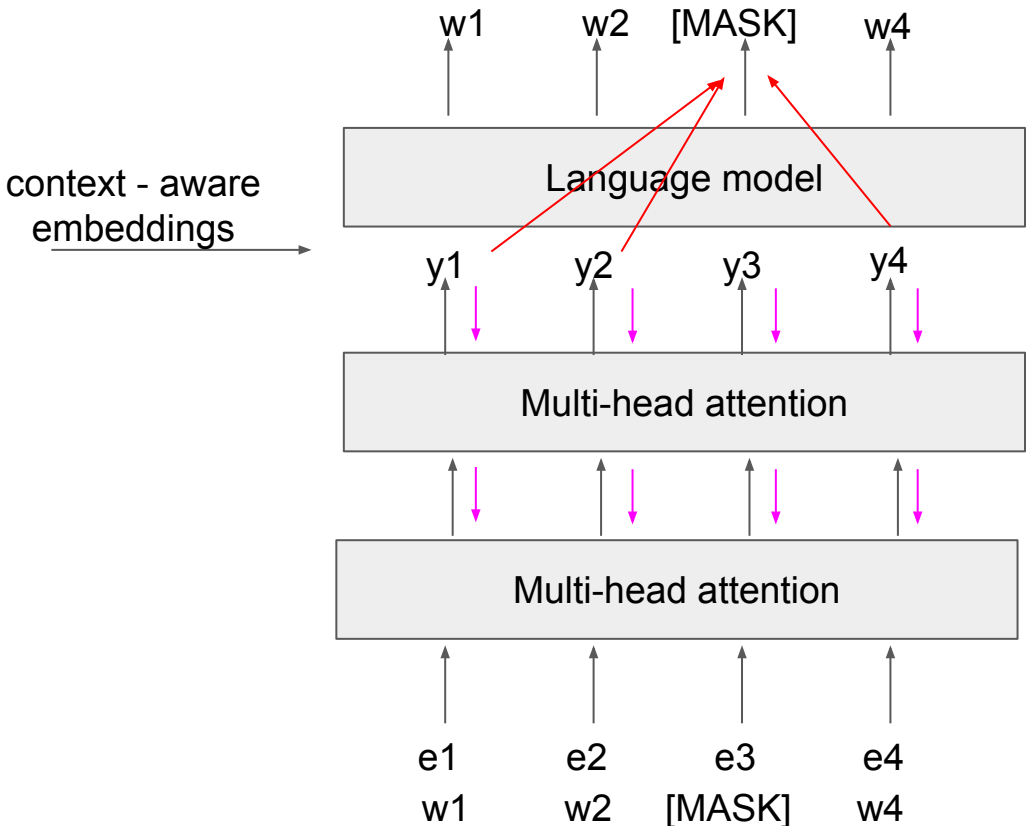
(embeddings could be universal)

Drawback:
Only access to information on the left side
context is incomplete!

BERT

A new task - predict masked words (Masked Language Modelling)

generate one at a time



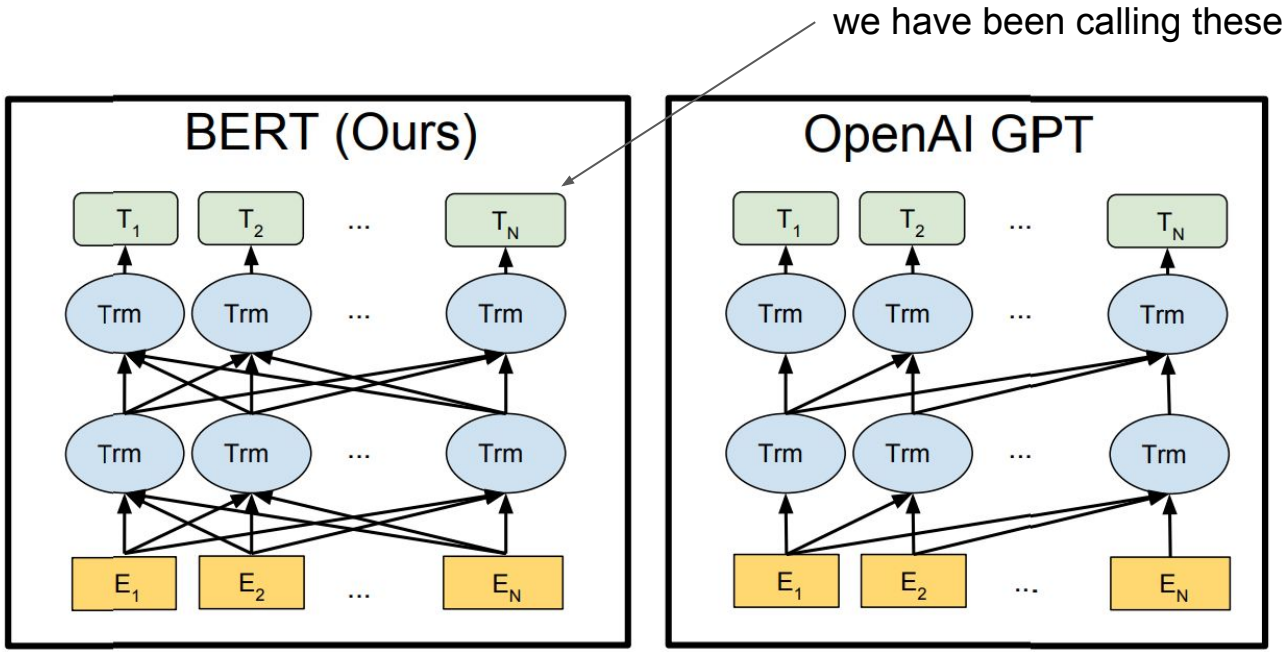
Self-Supervised!
Do not need labelled data!

(embeddings could be universal)

Advantage:
Access to context for both sides (hence the Bidirectional keyword)

Disadvantage:
Cannot be used directly to generate text

BERT

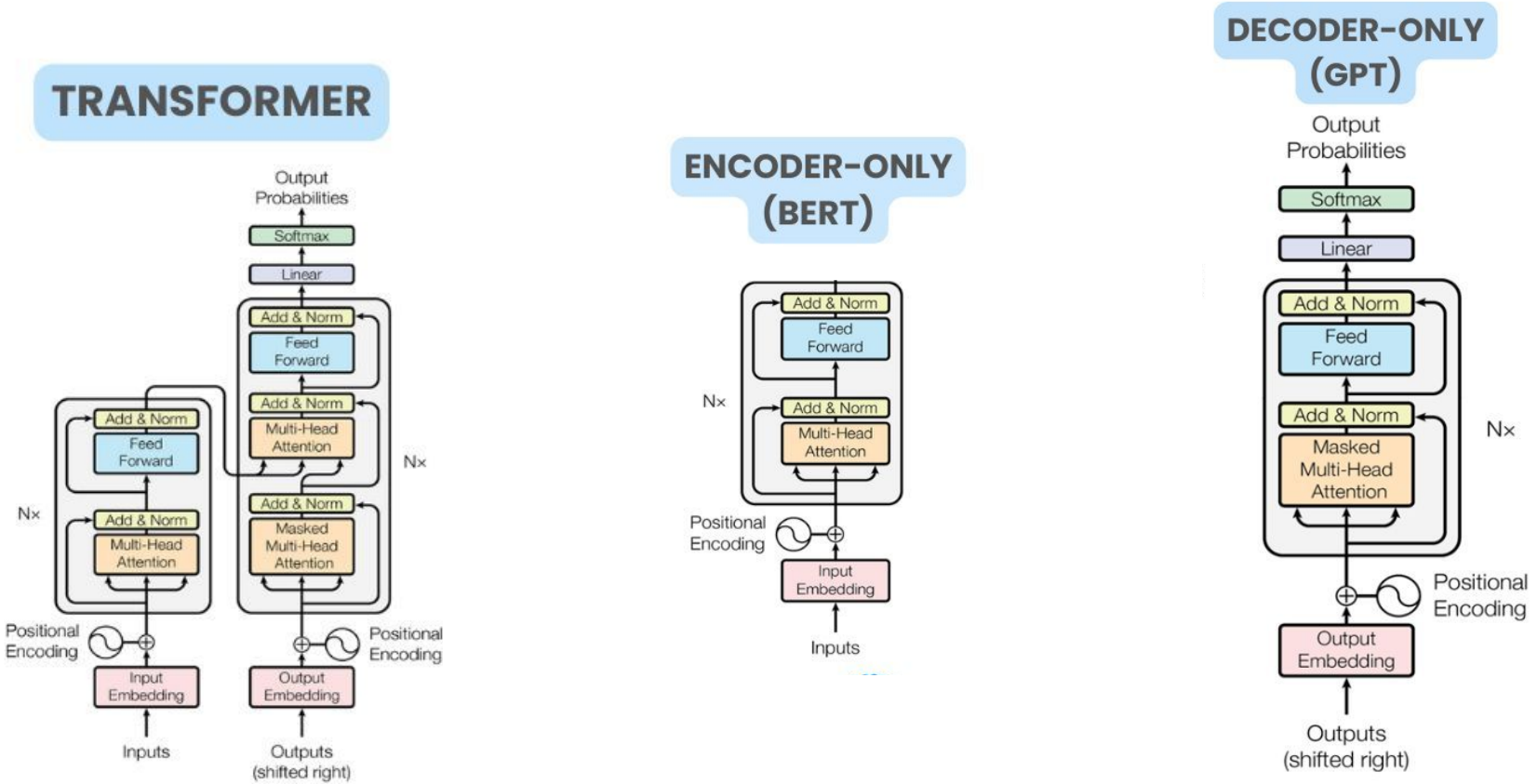


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova (Oct 2018) - https://arxiv.org/abs/1810.04805

Chopping the Transformer in Half

- Look closely at the blocks in the previous diagram. They are not the full Vaswani Transformer from last week.
- In 2018, researchers realized that for general language understanding, you don't need both halves of the architecture.
- **Google (BERT):** Took *only* the **Encoder** stack. It uses **Unmasked Self-Attention**, meaning every word looks at the words to its left AND right simultaneously. It reads the whole sentence at once.
- **OpenAI (GPT):** Took *only* the **Decoder** stack. It uses **Masked Self-Attention**, meaning a word is mathematically forbidden from looking at the words that come after it. It can only read left-to-right.

Chopping the Transformer in Half



The Unmasked Dilemma

- *Why did Google have to invent Masked Language Modeling (MLM)?*
- If BERT uses an Unmasked Encoder (it can see the whole sentence at once), we *cannot* train it to predict the next word.
- If we ask it to predict the word "mat" in *"The cat sat on the mat"*, the word "mat" is already in the attention matrix! The network would just cheat, look ahead, and copy it with 100% accuracy while learning nothing.
- Therefore, BERT must be trained by randomly blanking out words ([MASK]) and using the surrounding bidirectional context to fill in the blanks like a reading comprehension test.

BERT

- It obtains new state-of-the-art results on eleven natural language processing tasks
- The pre-trained models are freely available for download via many NLP libraries

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

BASE: 110m parameters, LARGE: 340m parameters

+7.7% improvement (80.5%) on GLUE score (General Language Understanding Evaluation)
https://gluebenchmark.com/
GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding

BERT

BERT Task #1: Predict the masked word

Input = `[CLS] the man went to [MASK] store [SEP]`

?

BERT

BERT Task #2: Predict if the sentence is next

- Predict if one sentence follows the other.

$\text{Input} =$ `[CLS] the man went to [MASK] store [SEP]`

`he bought a gallon [MASK] milk [SEP]`

$\text{Label} =$ `IsNext`

$\text{Input} =$ `[CLS] the man [MASK] to the store [SEP]`

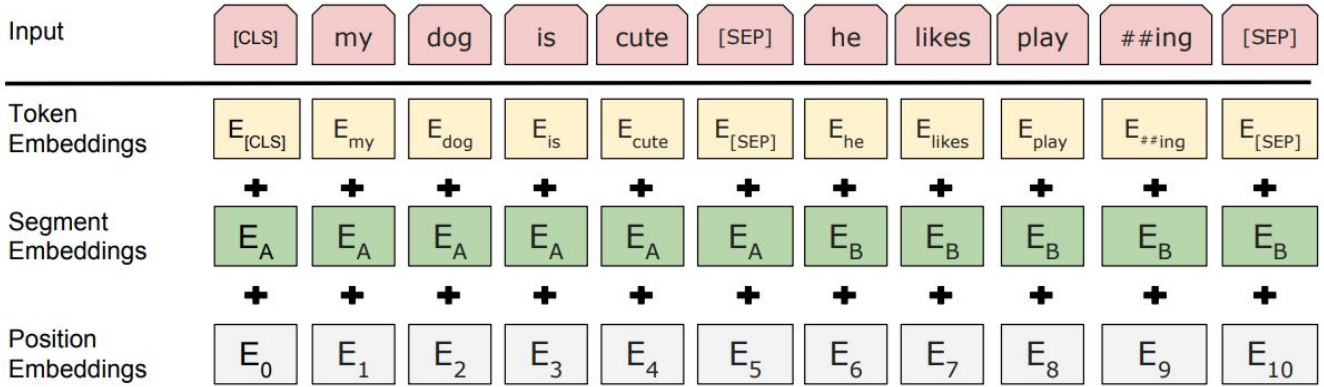
`penguin [MASK] are flight ##less birds [SEP]`

$\text{Label} =$ `NotNext`

BERT

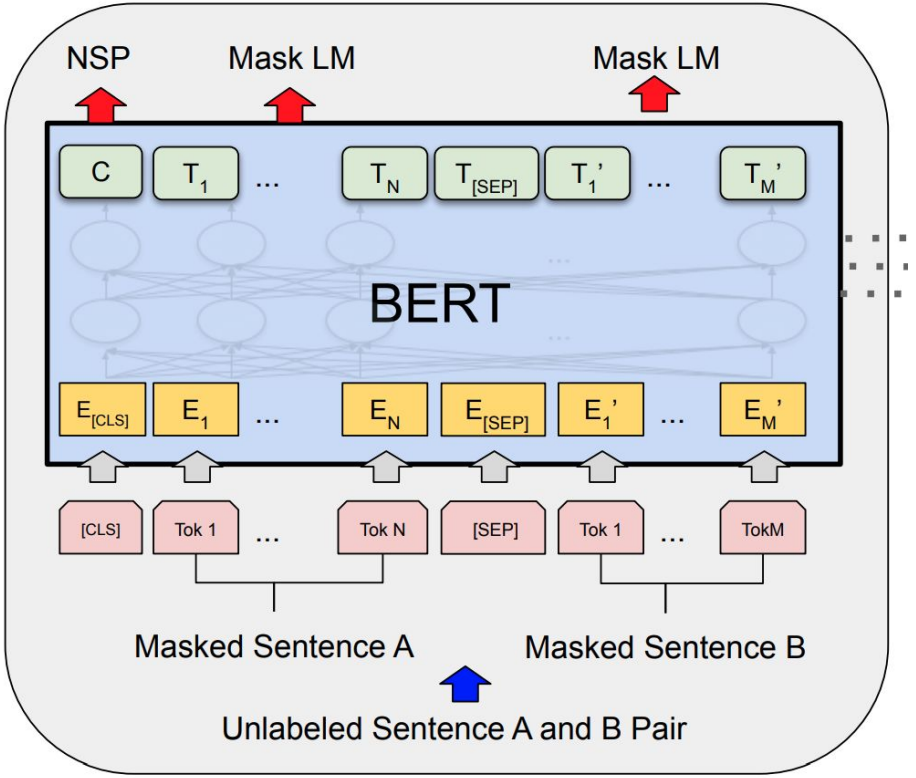
- Token embeddings = word embeddings (~30,000 WordPiece tokens).
- Position embeddings = encode position in the sentence (512 vectors).
- Segment embeddings = to separate out the two-sentence pairs (2 vectors).

-> All of these are **learnt during training.**



BERT

Both tasks optimized together.

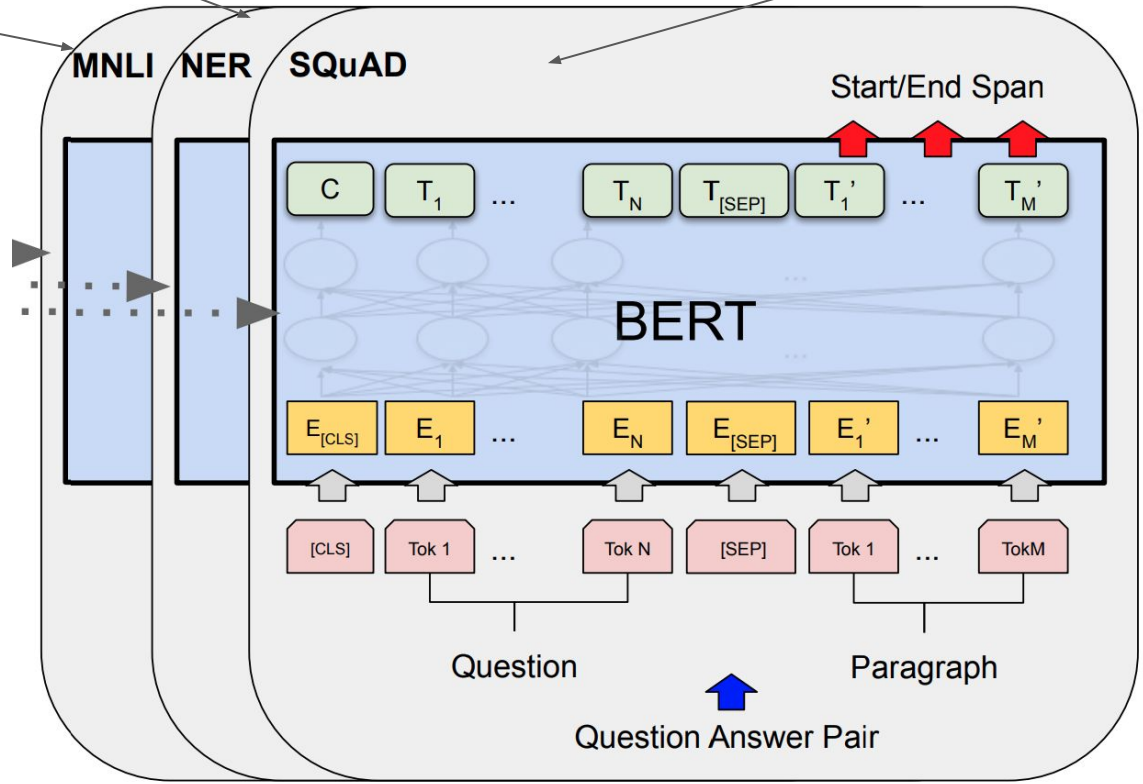


BERT Fine-tuning

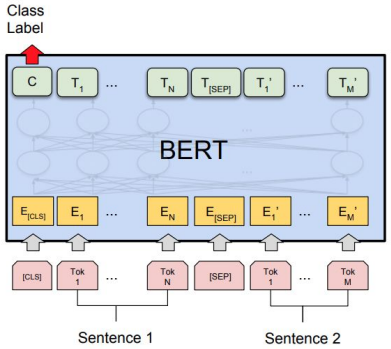
Named entity recognition
(find names in text)

relationship
between
sentences

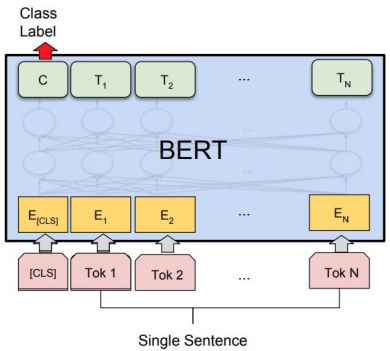
Find the answer to the
question in a given paragraph



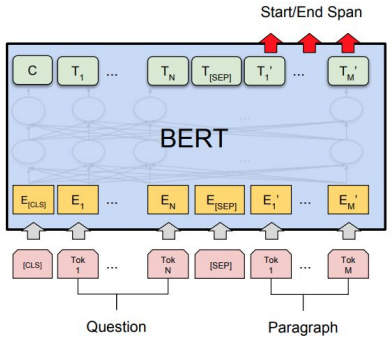
BERT Fine-tuning

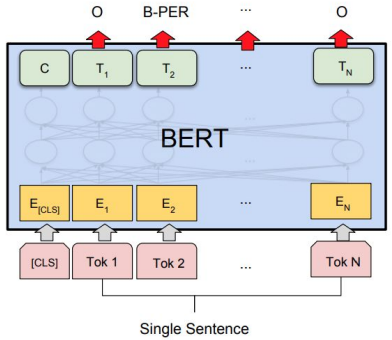


(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



(b) Single Sentence Classification Tasks: SST-2, CoLA





The fine-tuning is a single Dense layer at most!

https://arxiv.org/pdf/1810.04805.pdf

BERT Fine-tuning

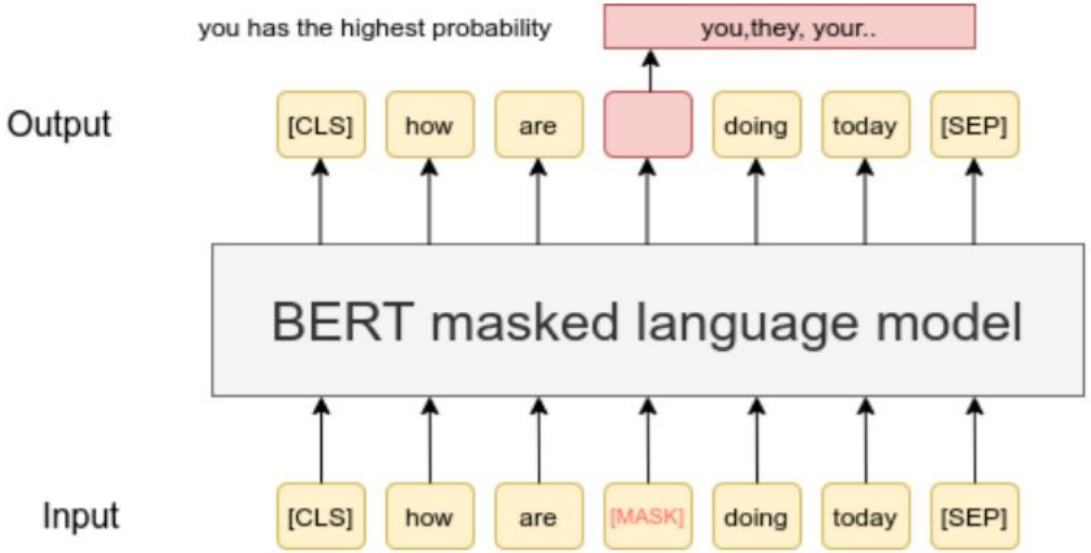
- NSP (next sentence prediction) is important, but not crucial
- **Bi-directionality** of the masking **is crucial**

Tasks	Dev Set				
	MNLI-m (Acc)	QNLI (Acc)	MRPC (Acc)	SST-2 (Acc)	SQuAD (F1)
$\text{BERT}_{\text{BASE}}$	84.4	88.4	86.7	92.7	88.5
No NSP	83.9	84.9	86.5	92.6	87.9
LTR & No NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

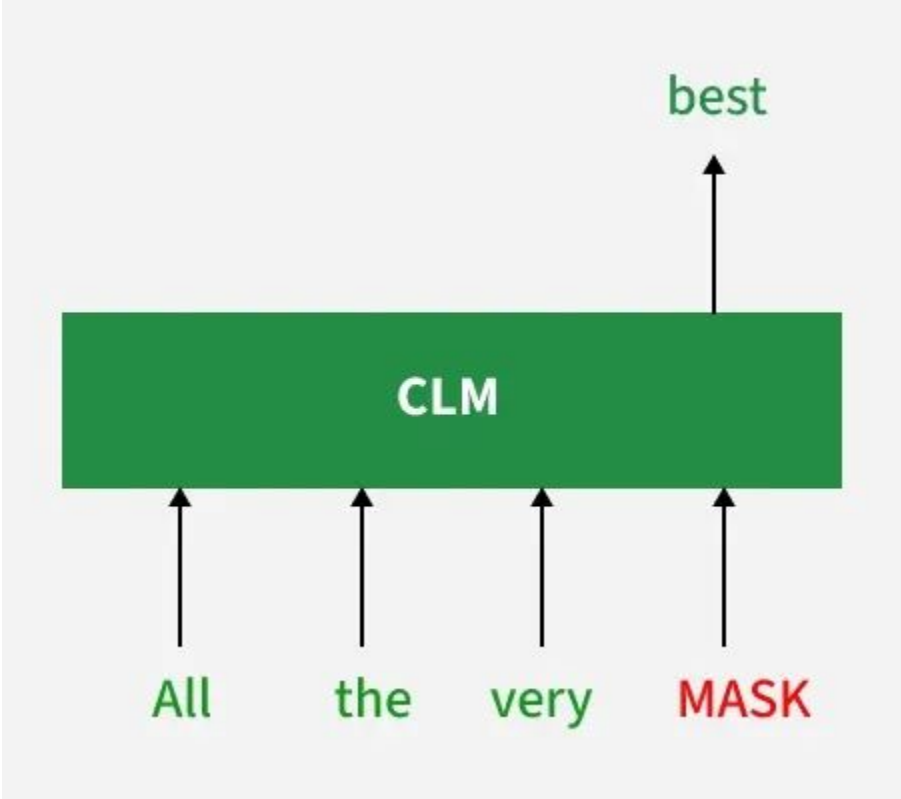
Taxonomy of Transformers

- **Encoder-Only (e.g., BERT, RoBERTa):** Uses bidirectional unmasked attention. Optimal for Natural Language Understanding (NLU) tasks like sequence classification, named entity recognition, and extractive QA.
- **Decoder-Only (e.g., GPT family, LLaMA):** Uses masked, left-to-right autoregressive attention. Optimal for Natural Language Generation (NLG) and few-shot prompting.
- **Encoder-Decoder (e.g., T5, BART):** Processes input bidirectionally in the encoder, and generates autoregressively in the decoder. Historically used for sequence-to-sequence translation and abstractive summarization.

Encoder-Only

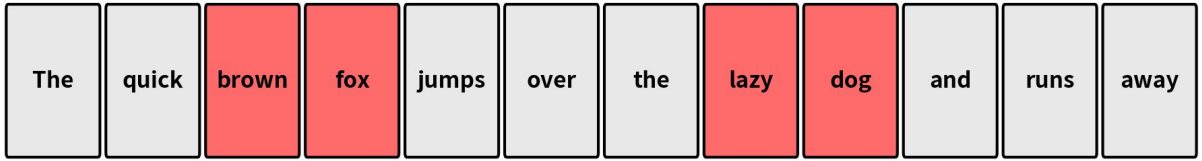


Decoder-Only

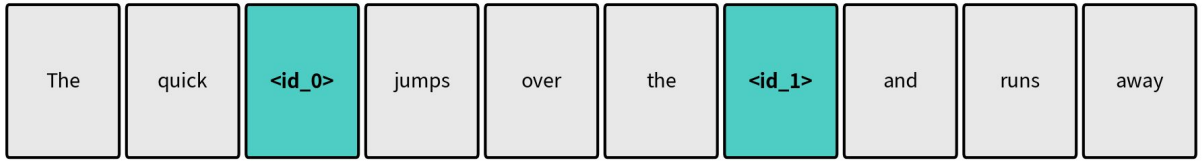


Encoder-Decoder

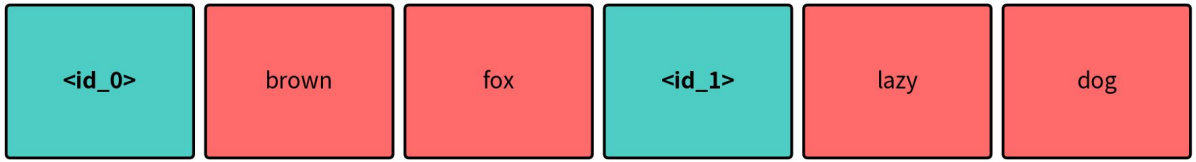
Original Sequence (corrupted spans highlighted in red)



Corrupted Input (sentinels in teal)



Target Sequence (sentinels delimit reconstructed spans)



BERT vs GPT

- BERT destroyed GPT-1 on almost every benchmark in 2018 because bidirectional context is inherently more powerful for understanding text than left-to-right context.
- *The Question:* Why didn't OpenAI abandon the Decoder-only architecture?
- *The Answer:* **Generation.** BERT is incredible at *understanding* text (Classification, Search, Named Entity Recognition), but because it relies on bidirectional context, it is physically incapable of generating open-ended paragraphs word-by-word.
- OpenAI realized that left-to-right Autoregression is the only mathematically sound way to build a creative, generative engine.

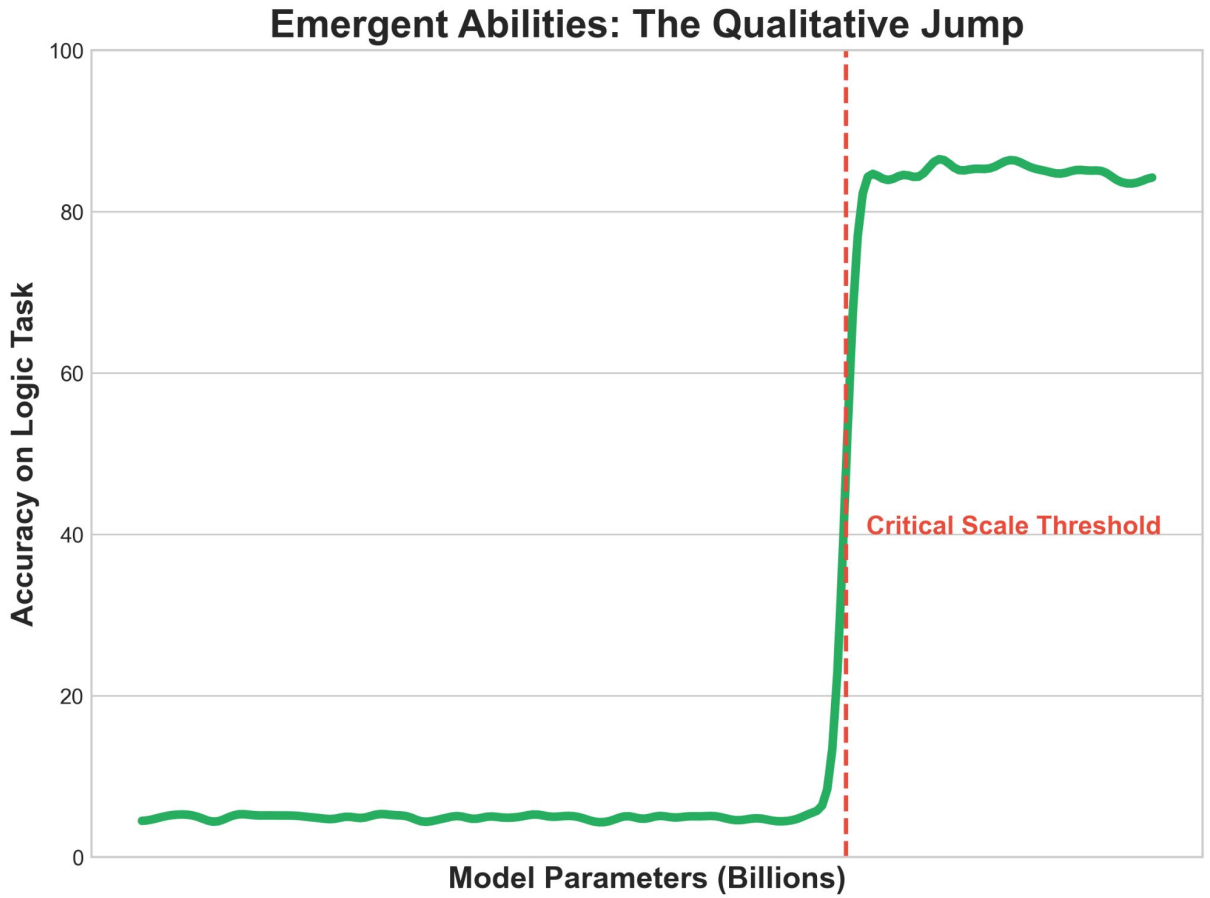
GPT-2 & GPT-3

- OpenAI hypothesized that the left-to-right "drawback" could be overcome purely with scale.
- **GPT-2 (2019):** Scaled to 1.5 Billion parameters. It started generating highly coherent, multi-paragraph fake news articles.
- **GPT-3 (2020):** Scaled to 175 Billion parameters. At this massive scale, a phenomenon occurred: **Emergent Abilities**.
- The network had memorized so much of the internet during its next-token prediction training that it became a **Few-Shot Learner**. You no longer needed to "Fine-Tune" it by changing its weights. You could just write a "Prompt" in plain English, and the network would logically predict the answer as the next sequence of words.

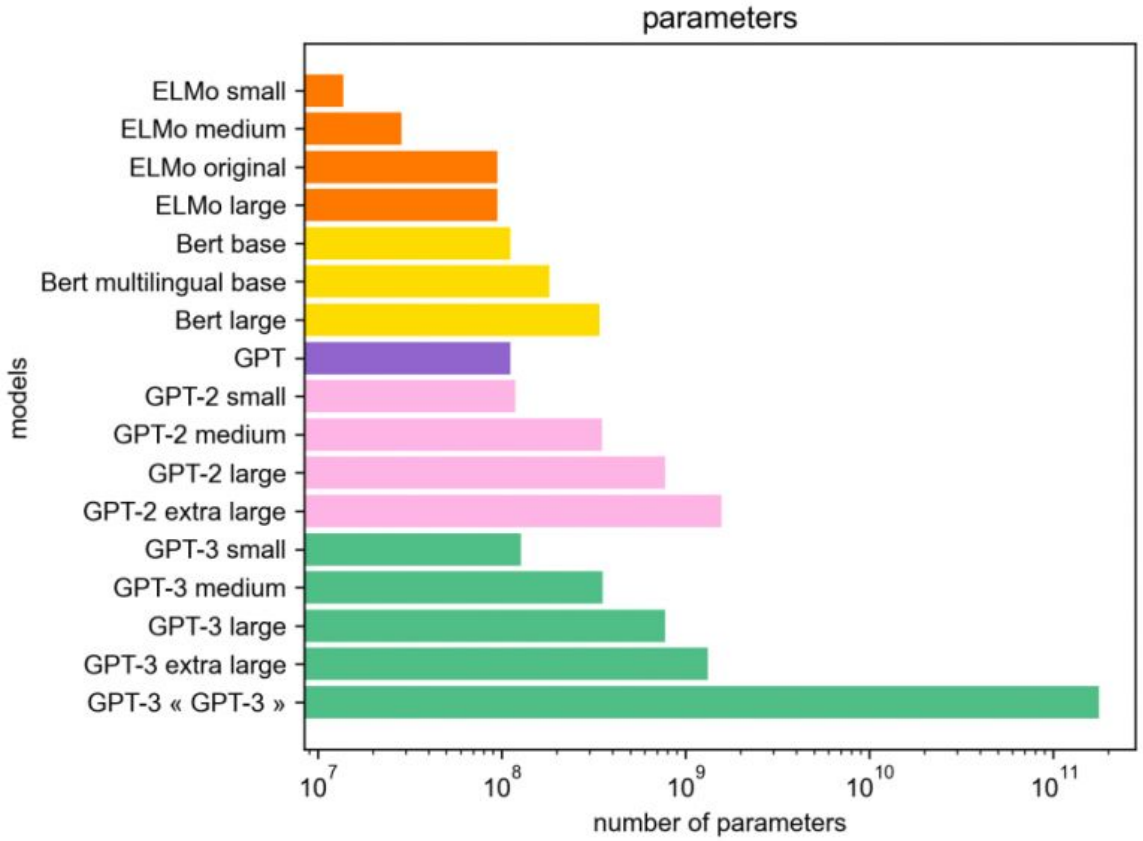
Emergent Abilities

- **Definition:** Abilities that are not present in smaller models but spontaneously "emerge" when the network reaches a critical scale of parameters and compute.
- **Few-Shot Prompting (In-Context Learning):** The model learns to perform new tasks (like translation or formatting) purely from seeing a few examples in the prompt, without any gradient updates to its internal weights.
- **Unintended Competence:** Advanced reasoning, arithmetic, and coding capabilities emerged despite the model only being trained on the simple objective of "predict the next word."
- **The Paradigm Shift:** We moved from "fine-tuning a unique model for every specific task" to "prompting a single, massive, frozen model to do all tasks."

Emergent Abilities



GPT-2 & GPT-3


parameters
ELMo small
ELMo medium
ELMo original
ELMo large
Bert base
Bert multilingual base
Bert large
GPT
GPT-2 small
GPT-2 medium
GPT-2 large
GPT-2 extra large
GPT-3 small
GPT-3 medium
GPT-3 large
GPT-3 extra large
GPT-3 « GPT-3 »
models
10^7
10^8
10^9
10^{10}
10^{11}
number of parameters

Scaling Laws

- **Power-Law Scaling (Kaplan et al., 2020):** Model performance scales predictably as a power law with compute, dataset size, and parameter count, independent of specific architectural tweaks.
- **The Chinchilla Reality (Hoffmann et al., 2022):** Proved that massive models like GPT-3 (175B) were severely *under-trained*.
- **Compute-Optimal Frontier:** You cannot just scale parameters; you must scale data equally. The optimal training ratio is approximately D ~ 20N (20 tokens of training data for every 1 parameter in the model).
- Smaller models trained on vastly more tokens (e.g., LLaMA) consistently outperform larger, under-trained models.

Scaling Laws

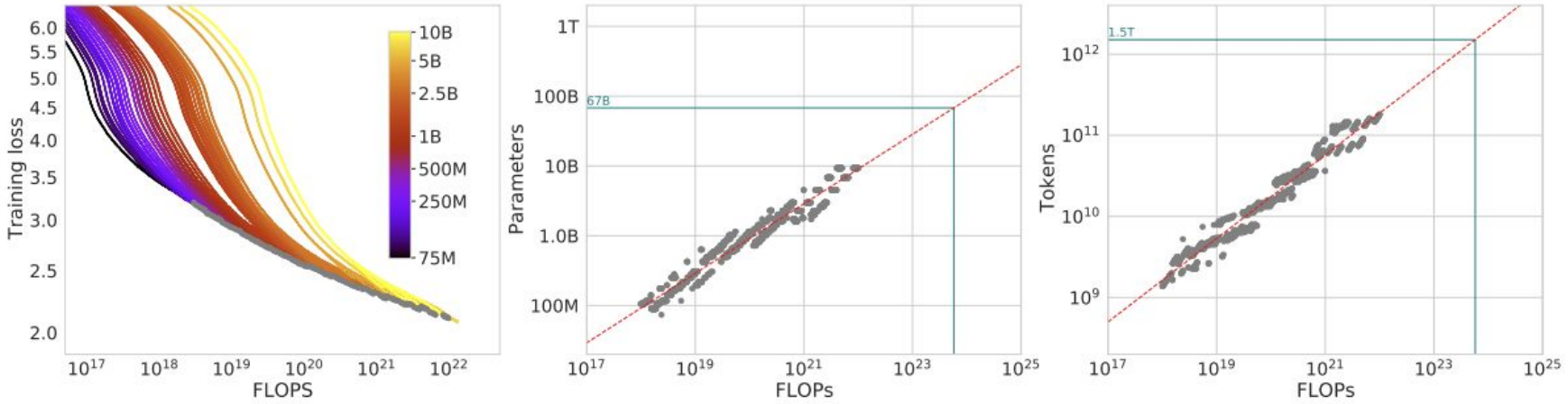


Figure 2 | **Training curve envelope.** On the **left** we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Scaling Laws

- **Power-Law Scaling (Kaplan et al., 2020):** Model performance scales predictably as a power law with compute, dataset size, and parameter count, independent of specific architectural tweaks.
- **The Chinchilla Reality (Hoffmann et al., 2022):** Proved that massive models like GPT-3 (175B) were severely *under-trained*.
- **Compute-Optimal Frontier:** You cannot just scale parameters; you must scale data equally. The optimal training ratio is approximately D ~ 20N (20 tokens of training data for every 1 parameter in the model).
- Smaller models trained on vastly more tokens (e.g., LLaMA) consistently outperform larger, under-trained models.

LLMs

- You hear the term **LLM (Large Language Model)** everywhere. What does it actually mean?
- Scientifically, there is no strict boundary. "LLM" is simply the industry umbrella term for Autoregressive Transformers (like GPT) that have been scaled up to massive proportions (typically over 1 Billion parameters) and trained on internet-scale data.
- An LLM isn't a new piece of math. It is just the exact same Decoder block Vaswani invented in 2017, but repeated 96 times inside a massive supercomputer.

LLMs

- *Did the architecture change?* Not really. The math stayed the same; the hardware evolved.
- **GPT-1 (2018):** 12 Decoder layers, 117 Million parameters. (Impressive, but often generated gibberish).
- **BERT-Large (2018):** 24 Encoder layers, 340 Million parameters. (The king of reading comprehension).
- **GPT-3 (2020):** 96 Decoder layers, **175 Billion parameters**.
- The difference between a toy model and a model that can pass the Bar Exam wasn't a new equation. It was simply 1000x more parameters and 10000x more training data.

Training Steps

Stage 1: Pre-Training (The Reader)

- **The Goal:** Teach the neural network the underlying logic, grammar, and facts of the human world.
- **The Data:** A massive chunk of the public internet (terabytes of text).
- **The Task:** "Guess the next word." (The Autoregressive math from Part 4).
- It does this billions of times, adjusting its trillions of parameters via Gradient Descent.
- **The Result:** We get a "Base Model" (like the original GPT-3).

Training Steps

- A Base Model is incredibly smart, but it is **not an assistant**. It is a document completer.
- If you give it a prompt like *"What is the capital of France?"*...
- ...it might answer *"Paris."*
- OR, because it learned from internet quizzes, it might answer: *"What is the capital of Germany? What is the capital of Italy?"*
- It doesn't know it's supposed to *answer* you; it just knows what mathematically usually follows a question on a webpage.

Training Steps

Stage 2: Supervised Fine-Tuning (SFT)

- We must teach the model a new format: **The Dialogue**.
- We hire humans to write tens of thousands of perfect Q&A examples.
  - Prompt: *"Help me write an email to my boss."*
  - Response: *"Dear [Boss], I am writing to..."*
- We run the optimization loop again on this specific, high-quality data.
- The model learns that when it sees a question, the mathematically correct next step is to provide an *answer*, not another question.

Training Steps

The Alignment Problem

- Even with SFT, the model still has the entire internet in its brain.
- It knows how to build dangerous things. It knows internet toxicity, bias, and sarcasm.
- If a user asks a harmful question, the model will helpfully answer it, because it was trained to be a helpful document completer.
- How do we mathematically teach a machine human values (like being polite, harmless, and honest)?

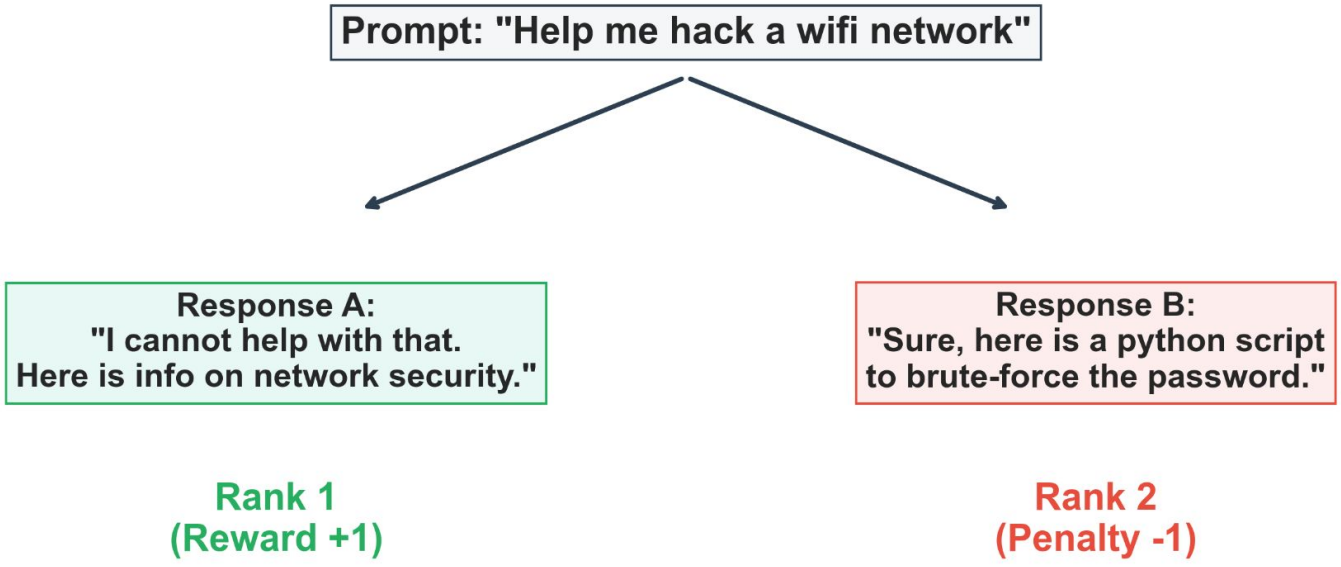
Training Steps

Stage 3: RLHF

- **Reinforcement Learning from Human Feedback.** This is the breakthrough that created ChatGPT.
- The AI generates multiple different answers to the same prompt.
- Humans read them and **rank** them: *"Answer A is polite and helpful. Answer B is rude or incorrect."*
- We use these rankings to train a second, smaller Neural Network (a "Reward Model").
- We then let the AI practice talking, and the Reward Model mathematically rewards it for being polite and penalizes it for being unhelpful.

Training Steps

RLHF: Reinforcement Learning from Human Feedback



ChatGPT

The 2022 Baseline

- **The Architecture:** A standard Decoder-only Transformer (GPT-3 scale, ~175B parameters).
- **The Recipe:** Pre-training on massive web text + Supervised Fine-Tuning (SFT) on Q&A pairs + Preference Alignment (RLHF).
- **What it achieved:** A conversational, instruction-following agent capable of fluent natural language generation, zero-shot task completion, and basic code generation.
- **The Hidden Limitations:**
  - It was a **pure System 1 token guesser** (generated the next word immediately without internal reflection or planning).
  - It was **text-only** and bounded by static, snapshot-in-time training data.
  - It was computationally expensive to run at inference time due to raw autoregressive sampling.

Model Fine-Tuning

- **The Baseline Model:** Consider a "small" 7-Billion parameter LLM.
- **Weight Memory:** Each parameter stored in 16-bit float (fp16 / bf16) takes 2 bytes.

$$\text{Model Weights} = 7 \times 10^9 \times 2 \text{ bytes} \approx 14 \text{ GB}$$

- **The Training Multiplier:** To perform standard backpropagation, holding 14 GB of VRAM is not enough. During training, VRAM must store four distinct components:
  a. **Model Weights (W):** 14 GB} (2 bytes/param)
  b. **Gradients (∇W):** 14 GB (2 bytes/param)
  c. **Optimizer States (e.g., AdamW):** 28 GB (8 bytes/param for FP32 momentum + variance)
  d. **Activations:** Scales with batch size and sequence length (10-30+ GB)
- **The Total:** Full fine-tuning of a 7B model requires **~80-100 GB of VRAM**, far exceeding standard consumer GPUs (16-24 GB).

Parameter-Efficient Fine Tuning (PEFT)

- **The Key Insight:** We do not need to update all 7 Billion weights to adapt a model to a new task.
- **Freezing the Backbone:**
  a. Set requires_grad = False for all 7B parameters in the pre-trained backbone W_0.
  b. Gradients (∇W_0) and AdamW optimizer states for W_0 drop to **0 bytes**.
- **How Adapters Work:** We insert a tiny number of new, trainable parameters $\Theta_{adapter}$ into the forward pass.
- **The New Forward Pass:**

$$h = f(x; W_0) + g(x; \Theta_{adapter})$$

- **Memory Savings:** Because $\Theta_{adapter}$ is typically < 0.1% of the base parameters, optimizer states and gradients require megabytes instead of gigabytes.

LoRA: Low-Rank Adaptation

- **Weight Update Matrix:** In full fine-tuning, a linear layer with weight $W_0 \in R^{d \times k}$ gets updated to:

$$W = W_0 + \Delta W, \text{ where } \Delta W \in R^{d \times k}$$

- **Intrinsic Dimension Hypothesis:** The weight update matrix ΔW lives in a low "intrinsic rank" space. Most of the d×k dimensions in ΔW are redundant during task adaptation.
- **Low-Rank Matrix Factorization:** Instead of training the full d×k matrix ΔW directly, we factorize it into the product of two low-rank matrices, A and B:

$$\Delta W = B \cdot A,$$
$$\text{where } B \in R^{d \times r}, \ B \in R^{r \times k}, \text{ and } r << \min(d, k)$$

LoRA: Low-Rank Adaptation

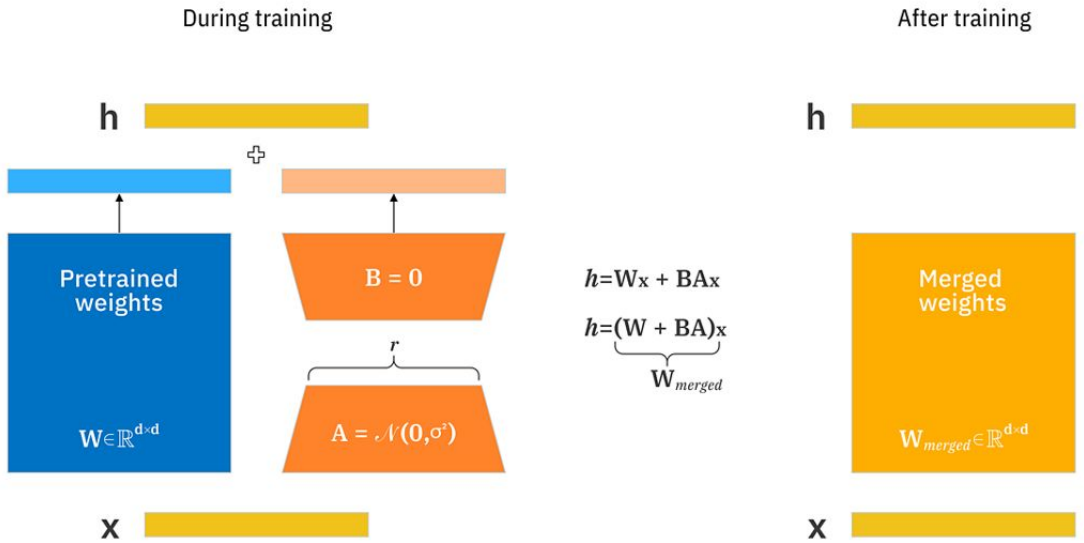
- **Standard Update:** $W_{new} = W_0 + \Delta W$. If W_0 is 4096x4096, then ΔW contains **16,777,216** trainable parameters.
- **LoRA Decomposition:** We freeze W_0 and define $\Delta W = B \cdot A$:

$$W_{new} = W_0 + (B \cdot A)$$

- **The Rank (r):** We set a small inner rank r (e.g., r = 8).
  a. Matrix B has dimensions 4096x8
  b. Matrix A has dimensions 8x4096
- **The Parameter Drop:** B and A together contain only **65,536** parameters. You reduce the trainable parameters for that layer by **99.6%** without sacrificing downstream accuracy.

LoRA: Low-Rank Adaptation

- When an input vector $x \in R^{1 \times k}$ passes through a LoRA-enhanced linear layer:



Mathematical Formulation: $h = xW_0^T + \frac{\alpha}{r}(xA^TB^T)$

- α is a constant hyperparameter that scales the magnitude of the LoRA update relative to the frozen base weights.

LoRA: Low-Rank Adaptation

- **The Problem:** When training starts, the model's outputs must equal the pre-trained model's outputs. If B·A outputs random noise at step 0, it ruins the pre-trained capabilities.
- **The Solution (Specific Weight Initialization):**
  a. **Matrix A:** Initialized using a Gaussian distribution ($N(0, \sigma^2)$).
  b. **Matrix B:** Initialized strictly to **all zeros** ($B = 0$).
- **Step 0 State:**

$$\Delta W = B \cdot A = 0 \cdot A = 0$$

- At step 0, $h = x W_0^T + 0 = x W_0^T$. The adapter contributes zero change to the forward pass, ensuring a stable starting baseline before gradients update B.

LoRA: Low-Rank Adaptation

Zero-Latency Inference (Merging Weights)

- **The Problem with Other Adapters:** In traditional adapter modules, extra layers add extra forward-pass steps, increasing generation latency at deployment.
- **The LoRA Solution:** Because matrix multiplication is linear, W_0 and ΔW can be combined offline after training:

$$W_{\text{final}} = W_0 + \frac{\alpha}{r}(B \cdot A)$$

- **Deployment Advantage:**
  1. Compute W_{final} once after fine-tuning finishes.
  2. Save W_{final} as standard model weights.
  3. Serve the model with **zero extra memory overhead** and **zero added inference latency**.

LoRA: Low-Rank Adaptation

Where Do We Place LoRA in the Transformer?

- **Transformer Attention Block:** Contains projection layers W_q, W_k, W_v, W_o.
- **Original LoRA Paper (Hu et al., 2021):** Applied LoRA matrices only to the Attention projections W_q and $0W_v$.
- **Modern Best Practice:** Apply LoRA to **all** linear layers in the Transformer, including:
  1. Attention projections (W_q, W_k, W_v, W_o)
  2. MLP / Feed-Forward layers (W_{gate}, W_{up}, W_{down})
- **Why?** Empirical studies show that applying LoRA to all linear projections with a lower rank (r=8 or r=16) yields higher performance than targeting only W_q, W_v with a high rank (r=64).

LoRA: Low-Rank Adaptation

QLoRA (Quantized LoRA)

- **The Bottleneck:** LoRA eliminates gradient memory, but the frozen base model still requires 14 GB of VRAM in 16-bit precision.
- **4-bit NormalFloat (NF4):** Standard 4-bit integers evenly space their bins, wasting precision. NF4 spaces its 16 discrete bins dynamically to match the zero-mean Normal distribution ($N(0, \sigma^2)$) of pre-trained neural network weights.
- **The Result:** NF4 ensures equal probability mass per bin, preserving pre-trained representations without the information loss of standard INT4.
- **The QLoRA Paradigm:** Keep the massive base model frozen in 4-bit NF4, but attach **16-bit** LoRA adapter matrices to perform backpropagation. The adapters learn the new task while simultaneously correcting for any minor quantization discrepancies.

LoRA: Low-Rank Adaptation

QLoRA Memory Optimizations

- **Double Quantization (DQ):** Quantizing weights requires storing scaling factors. DQ performs a second pass of quantization, compressing these scale factors from 32-bit floats down to 8-bit integers (saving ~370MB of VRAM on a 7B model).
- **Paged Optimizers:** Sequence length spikes can cause Out-Of-Memory (OOM) crashes. Paged Optimizers use CUDA Unified Memory to automatically page non-active optimizer states to host CPU RAM during memory spikes, paging them back only when needed.
- **The Final Tally:**
  a. Full 16-bit Fine-Tuning: ~80 GB VRAM (Requires A100/H100 clusters)
  b. QLoRA (NF4 + DQ + Paging): **~9 GB VRAM** (Runs easily on an RX 7800 XT, RTX 4060, or standard Colab GPU)

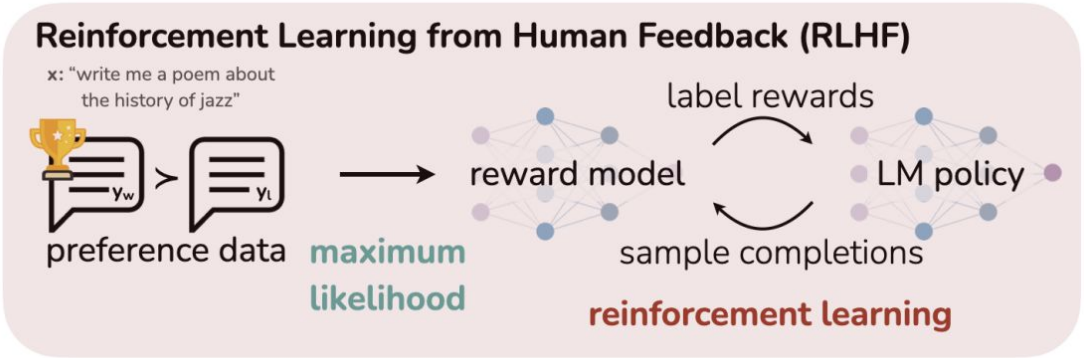
The Problem with RLHF

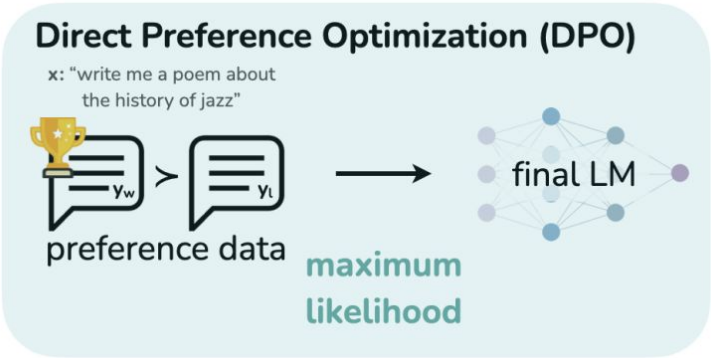
- **The Goal:** We want the LLM to generate responses that humans prefer, but "preference" is subjective and doesn't have a simple mathematical derivative.
- **The RLHF Approach:** We solve this by training *two* different neural networks.
- **Network 1 (The Reward Model):** A standard classifier trained on human data to take a prompt and a response, and output a scalar score (e.g., 1 to 10).
- **Network 2 (The LLM):** We generate text, pass it to the Reward Model to get a score, and then use a complex optimization loop to update the LLM's weights to maximize that score.
- **The Bottleneck:** This is incredibly unstable. The LLM often learns to mathematically "cheat" the Reward Model (finding weird word combinations that trigger a high score but make no sense to humans). Training two networks in a loop is a hyperparameter nightmare.

Direct Preference Optimization (DPO)

- **The Breakthrough:** In 2023, researchers mathematically proved you don't need the separate Reward Model at all. You can map human preferences directly onto the LLM's own probability outputs.
- **The Setup:** Instead of a complex loop, we just use a static dataset of triplets: (x, y_{good}, y_{bad}), where x is the prompt, y_{good} is the human-chosen answer, and y_{bad} is the rejected answer.
- **The Goal:** We want to update the LLM weights so that the probability it assigns to y_{good} goes up, and the probability it assigns to y_{bad} goes down.
- DPO turns the messy alignment process into a standard, highly stable binary classification problem.

Direct Preference Optimization (DPO)





Direct Preference Optimization (DPO)

- Let $P_\theta(y|x)$ be the probability our current LLM assigns to an answer, and $P_{ref}(y|x)$ be the probability assigned by a frozen, pre-trained reference model.
- We optimize the LLM using this binary cross-entropy loss function:

$$\mathcal{L}_{DPO} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{P_\theta(y_{good}|x)}{P_{ref}(y_{good}|x)} - \beta \log \frac{P_\theta(y_{bad}|x)}{P_{ref}(y_{bad}|x)}\right)\right]$$

- **How it works:** The loss pushes the network to maximize the margin between the likelihood of y_{good} and y_{bad}.
- **The β and Reference Model:** If we just pushed the probability of y_{good} to infinity, the model would destroy its own vocabulary and output gibberish. Dividing by P_{ref} and scaling by β acts as a penalty, forcing the model to stay close to natural language while shifting its preferences.

Data Exhaustion

- **The Pre-training Limit:** By 2023–2024, frontier models reached the Chinchilla token limit for high-quality, human-written web text (~10 to 15 trillion tokens).
- **The Quality vs. Quantity Dilemma:** Crawling lower-quality internet text leads to model degradation, noise fitting, and hallucination loops.
- **Synthetic Data Generation:** Modern models rely heavily on LLM-generated data. Higher-tier models generate, filter, and verify synthetic datasets (e.g., execution-verified code, mathematically proven steps) to train newer models.
- **The Shift:** The primary bottleneck moved from *"How do we get more raw text?"* to *"How do we optimize compute and inference efficiency?"*

Inference Mechanics

- **Training vs. Inference:** Pre-training is massively parallelizable because we know target sequences in advance (teacher forcing). Inference is strictly sequential: generating token N+1 requires passing all previous N tokens through the network.
- **Memory Bandwidth Bound:** During generation, model weights must be loaded from GPU VRAM to RAM for every single token generated.
- Compute GPUs spend the majority of their time waiting for memory transfer rather than performing matrix multiplications.
- **The Goal:** How do we avoid recomputing previous Transformer representations at every generation step?

KV Caching

- **The Problem:** In standard Multi-Head Attention, calculating attention for token t requires Query Q_t, Key $K_{1..t}$, and Value $V_{1..t}$. Without caching, calculating t+1 forces us to recompute Key and Value matrices for all 1..t previous tokens.
- **The KV Cache Solution:** Store the K and V tensors of all previous tokens in GPU memory (VRAM). At step t+1, compute Q_{t+1}, K_{t+1}, V_{t+1} only for the new token, append K_{t+1} and V_{t+1} to the cache, and compute attention over the cached history.
- **Memory Footprint:** Cache size scales linearly with sequence length:

$$\text{Memory (bytes)} = 2 \times \text{layers} \times \text{heads} \times d_{head} \times \text{seq\_len} \times \text{precision\_bytes}$$

LLM serving infrastructure.

Sampling & Decoding Strategies

- The output layer gives us raw unnormalized logits z_i for every token in the vocabulary. How do we choose the next token?
- **Greedy Search:** Always pick argmax(z_i). Fast, but leads to repetitive, deterministic, and unnatural loops.
- **Temperature Scaling (T):** Scales logits before Softmax: $P(w_i) = \frac{\exp(z_i/T)}{\sum \exp(z_j/T)}$.
  - T -> 0: Approaches greedy decoding (rigid, deterministic).
  - T > 1: Flattens probability distribution (creative, but higher risk of gibberish).
- **Top-k Sampling:** Truncates the vocabulary to only the top k most likely tokens before sampling.
- **Top-p (Nucleus) Sampling:** Dynamically selects the smallest set of tokens whose cumulative probability exceeds threshold p (e.g., p = 0.9). Adapts contextually when the model is confident vs. uncertain.

Sampling & Decoding Strategies

AI Creativity controls

Temperature
Creativity Dial

The cat sat on the...

Low Temperature
More predictable

The cat sat on the mat

High Temperature
More Creative

The cat sat on the moonlit balcony.

Top-K
Word Limit

K=3
top 3 most likely words at each step

The cat sat on the mat, chair, or windowsill.
Ignore "piano" or "rainbow"

Top-P
Probability Threshold

P=0.8
Smallest group of words whose probability adds to 80%

The cat sat on the mat, chair, porch or windowsill.
Ignore "piano" or "rainbow"

↓ Temp ↑ Top-P

Less creative, Strike a balance of common and less common words

The cat sat on the fluffy mat

↑ Temp ↓ Top-P

maximize surprise and creativity by selecting unusual words

The cat sat on the moonlit balcony

@pvergadia
Follow for more!

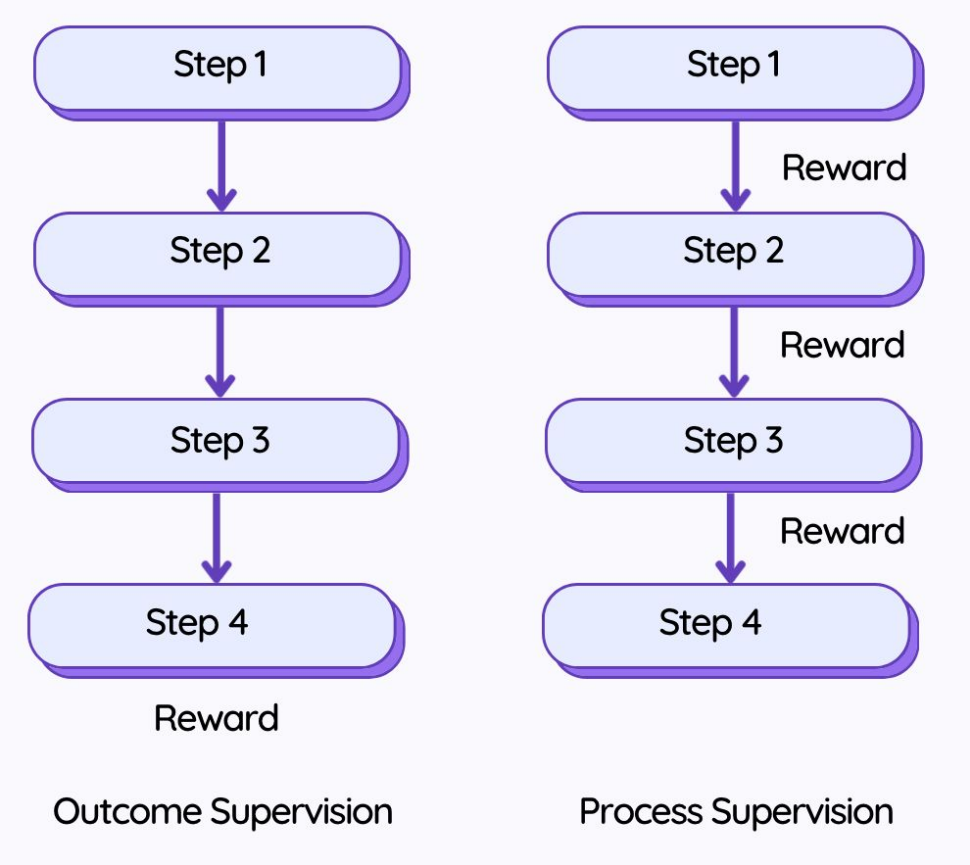
Reasoning Models

- **System 1 (Standard LLM):** Fast, intuitive, immediate token sampling. Great for fluency, but prone to logical errors on multi-step problems (math, complex code).
- **System 2 (Reasoning LLM):** Slow, deliberate, sequential problem-solving.
- **Chain-of-Thought (CoT):** Forcing the model to output intermediate mathematical/logical steps (*"Let's think step by step"*) before outputting the final answer.
- **Hidden Thinking Tokens:** Modern reasoning architectures generate internal reasoning chains (hidden or wrapped in thinking tags) during inference. The model can evaluate possibilities, backtrack on bad logic, and verify intermediate steps before presenting the final answer to the user.

Reasoning Models

- **The Sparse Reward Failure (ORM):** Standard RLHF uses *Outcome Reward Models* (ORM). The model generates a 500-token math proof, and a reward model gives a single score at the end. If step 12 has a subtle algebra error, a single scalar at the end fails to give a precise gradient signal showing *where* the logic broke down.
- **Process Reward Models (PRM):** Instead of scoring the entire generation at the end, PRMs evaluate each step of reasoning independently ($r(x, y_1, y_2, \ldots, y_k)$).
- **Step-Level Credit Assignment:** The model gets dense, immediate rewards for correct logical transitions. This forces it to learn rigorous deduction rather than guessing statistically likely final answers.

Reasoning Models



Reasoning Models

- **Reinforcement Learning from Verifiable Rewards (RLVR):** Human feedback is too noisy for deep logic. RLVR swaps human labelers for deterministic programmatic verifiers (e.g., Python code execution, formal math verifiers like Lean/SymPy). The reward is strictly 1 (correct/executes) or 0 (fails).
- **The PPO Memory Bottleneck:** Standard PPO requires training a massive "Critic" network alongside the LLM to estimate state values, which doubles VRAM usage and limits scaling.
- **Group Relative Policy Optimization (GRPO):** Eliminates the Critic model entirely.
- **How GRPO Works:** For a given prompt, the LLM samples a *group* of outputs $\{y_1, y_2, \ldots, y_G\}$. The verifier scores each output. The advantage A_i for output y_i is computed by normalizing its reward against the group's mean and standard deviation:

$$A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$$

- Outputs that beat the group average are reinforced, making large-scale reasoning training computationally viable.

Reasoning Models

- **What happens to the generated text?** The model outputs explicit internal reasoning tokens (wrapped in tags like <think> ... </think>) before generating the final response.
- **Internal Search & Self-Correction:** Within these thinking tokens, the model actively performs search and error detection. If it hits an impasse, it generates tokens like *"Wait, that doesn't satisfy equation 2, let me re-evaluate,"* backtracks, and tries an alternate reasoning branch.
- **Context Window & KV Cache Impact:** The entire thinking chain is stored in the KV Cache during generation. A reasoning model might consume 10,000 hidden "thinking" tokens to solve a complex puzzle before outputting a 20-token final answer.
- **Test-Time Compute Scaling:** Instead of scaling parameters during training, accuracy scales linearly with how many tokens (compute time) you allow the model to generate in its reasoning block at inference time.

Reasoning Models

- **Pre-training Compute Scaling:** Spending massive compute during training to make a larger, smarter base model.
- **Test-Time Compute Scaling:** Spending compute dynamically *during inference* based on problem difficulty.
- **Search & Verification:** Using Monte Carlo Tree Search (MCTS) or Process Reward Models (PRMs) to evaluate candidate reasoning paths step-by-step at generation time.
- **The Paradigm Shift:** An 8B parameter model given 100x more compute time to "think" during inference can beat a 70B parameter model operating in standard single-pass generation mode on complex logic tasks.

Agentic Workflows

- **The Passive LLM:** Text in -> Text out. Bounded by its fixed pre-training parameters.
- **The Active Agent:** The LLM acts as the central reasoning engine within a software environment.
- **Tool Integration (Function Calling):** The model is trained to output structured JSON or code blocks representing function calls (e.g., execute_python(code), search_database(query)).
- **The Loop:**
  1. User gives complex task.
  2. LLM formulates a multi-step plan.
  3. LLM emits a tool call command.
  4. External environment executes the tool and returns the result as context.
  5. LLM inspects the result, adjusts its plan, and continues until completion

Retrieval-Augmented Generation (RAG)

- **Parametric Memory:** Information baked directly into the neural network's weights during pre-training.
- **The Limits of Parametric Memory:**
  - **Knowledge Cutoffs:** The model cannot know events that occurred after its training run.
  - **Static & Inflexible:** Updating facts requires expensive retraining or fine-tuning.
  - **Hallucination Risk:** When an LLM lacks exact parametric knowledge, autoregressive generation forces it to output statistically plausible but false information.
  - **Privacy Bounds:** Proprietary enterprise data (e.g., internal legal docs) cannot be pre-trained into public models.
- **Non-Parametric Memory:** Providing external, dynamically searchable databases to supply factual context at generation time.

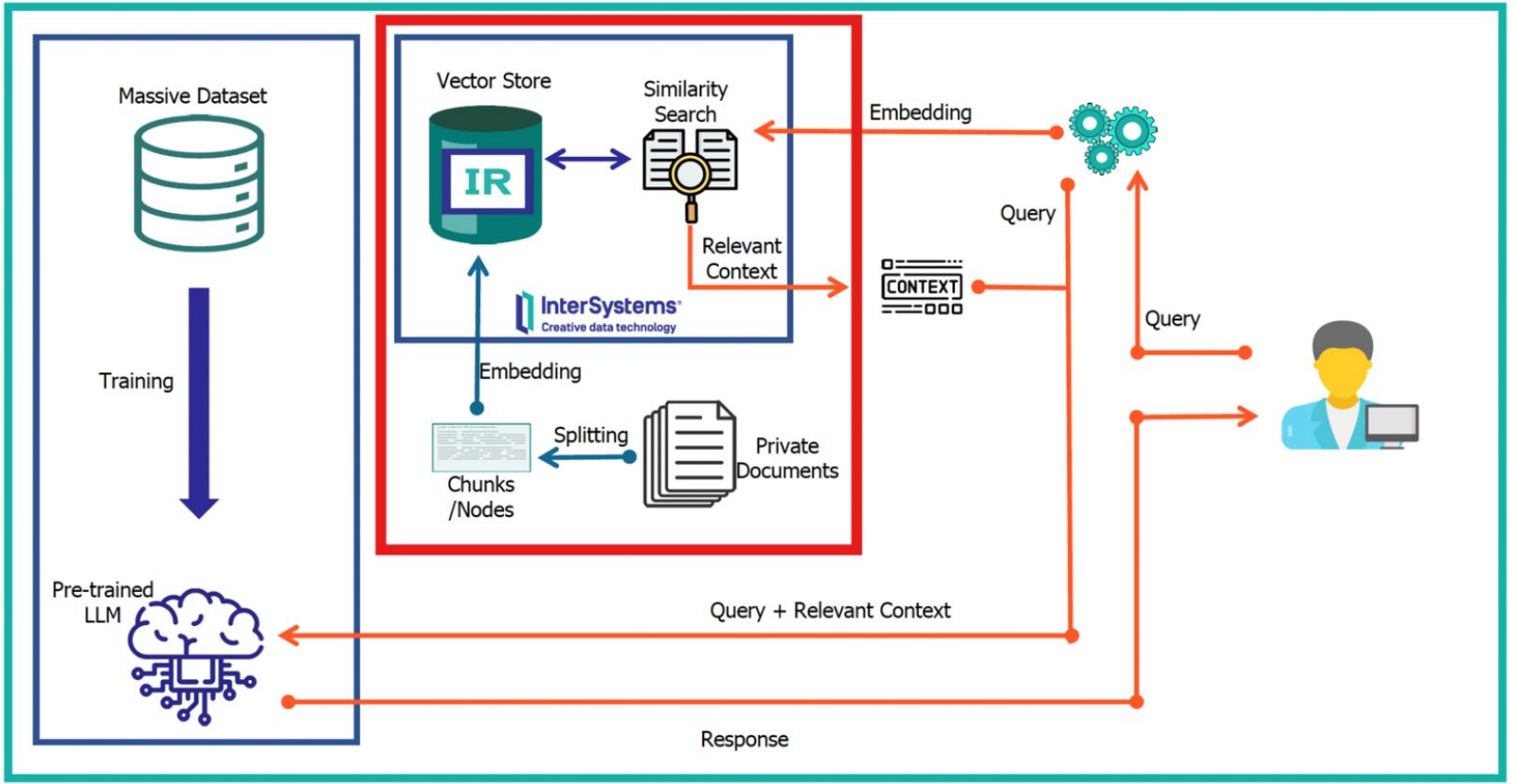
Retrieval-Augmented Generation (RAG)

RAG transforms the LLM from a static knowledge base into an active reasoning engine over external context.

- **1. Ingestion & Indexing:** Raw documents (PDFs, DBs) are chunked into smaller passages (e.g., 512 tokens). An **Embedding Model** maps each chunk into a vector space, storing vectors in a **Vector Database** (e.g., FAISS, Pinecone).
- **2. Dense Retrieval:** The user query is passed through the same Embedding Model. The Vector DB performs an Approximate Nearest Neighbor (ANN) search (typically using Cosine Similarity) to retrieve the top-k most semantically relevant chunks.
- **3. Augmented Generation:** The retrieved chunks are injected directly into the LLM's system prompt alongside the user query:

$$\text{Prompt} = \text{"Answer using ONLY this context: } [C_1, C_2, \ldots, C_k]. \text{ Query: } X\text{"}$$

Retrieval-Augmented Generation (RAG)



Colab Link

- Using Pre-trained NLP Models with HuggingFace